

جزوه ساختمان داده

وجیهه ده دله

۱۲ فروردین ۱۴۰۴

فهرست مطالب

۲	۱	مرتبۀ اجرایی الگوریتم ها
۲	۱.۱	ارزیابی مرتبه اجرایی یک الگوریتم (تابع زمانی)
۵	۲.۱	$BigOmega \Omega$
۸	۳.۱	$BigO O$
۱۰	۴.۱	$BigTheta \Theta$
۱۳	۲	آرایه ها
۱۳	۱.۲	انواع آرایه ها
۱۶	۲.۲	نحوه ذخیره سازی عناصر در آرایه
۱۹	۳.۲	عملیات روی آرایه ها
۲۴	۴.۲	ماتریس ها Matrix
۲۸	۳	زیر برنامه های بازگشتی
۲۸	۱.۳	تابع بازگشتی recursive
۳۵	۲.۳	کاربرد پشته در زیر برنامه های بازگشتی
۳۷	۴	صف queue
۳۸	۱.۴	درج در صف
۳۸	۲.۴	حذف از صف
۳۹	۵	پشته Stack
۳۹	۱.۵	درج در پشته
۴۰	۲.۵	حذف از پشته
۴۱	۳.۵	نگارش مختلف عبارت های ریاضی

۴۵	لیست پیوندی	۶
۴۷	۱.۶ لیست پیوندی یک طرفه	
۵۰	۲.۶ لیست پیوندی دو طرفه	
۵۲	۳.۶ لیست پیوندی حلقوی	
۵۳	۴.۶ پیاده سازی پشته با لیست پیوندی	
۵۴	۵.۶ پیاده سازی صف به کمک لیست پیوندی	
۵۵	درخت	۷
۵۵	۱.۷ تعاریف اولیه	
۵۹	۲.۷ پیاده سازی درخت دودویی	
۵۹	۱.۲.۷ پیاده سازی درخت دودویی با استفاده از آرایه	
۶۰	۲.۲.۷ پیاده سازی درخت با لیست پیوندی	
۶۱	۳.۷ پیمایش درخت دودویی	
۶۱	۱.۳.۷ روش های معمول پیمایش درخت دودویی	
۶۳	۲.۳.۷ الگوریتم های پیمایش درخت دودویی	
۶۴	۴.۷ رسم درخت دودویی	
۶۴	۱.۴.۷ رسم درخت دودویی با داشتن پیمایش های پیشوندی و میانوندی	
۶۶	۲.۴.۷ مشخص کردن گره های تک فرزندی	
۶۶	۳.۴.۷ درخت عبارت	
۶۷	۴.۴.۷ تعداد درخت های دودویی که می توان به n گره ساخت	
۶۷	۵.۴.۷ رسم درخت دودویی با داشتن پیمایش های پیشوندی و پسوندی	
۶۷	۵.۷ درخت کلی (عمومی)	
۶۸	۱.۵.۷ مراحل تبدیل یک درخت به درخت دودویی معادل	
۶۹	۶.۷ جنگل Forest	
۶۹	۱.۶.۷ تبدیل جنگل به درخت دودویی	
۷۱	۷.۷ پیاده سازی الگوریتم های مربوط به درخت	
۷۱	۱.۷.۷ ساختار یک گره یک درخت دودویی	
۷۱	۲.۷.۷ تابع newnode	
۷۱	۳.۷.۷ شمارش تعداد گره ها	
۷۲	۴.۷.۷ تشخیص برگ بودن	
۷۲	۵.۷.۷ تشخیص کامل بودن درخت دودویی	
۷۳	۶.۷.۷ حذف گره های تک فرزندی	
۷۳	۷.۷.۷ پیاده سازی درخت در صف	
۷۴	گراف	۸
۷۶	۱.۸ نمایش گراف	
۷۸	۲.۸ پیمایش در گراف ها	
۸۳	۳.۸ پیاده سازی گراف	

۸۷	مرتب سازی	۹
۸۸	مرتب سازی حبابی	۱.۹
۸۹	مرتب سازی انتخابی	۲.۹
۹۰	مرتب سازی درجی	۳.۹

۱. مرتبه اجرایی الگوریتم ها

فاکتورهای مهم در ارزیابی یک الگوریتم:

۱. میزان حافظه مصرفی

۲. زمان اجرای الگوریتم (محاسبه زمان اجرای الگوریتم)

در ارزیابی یک الگوریتم دو فاکتور مهمی که باید مورد توجه قرار گیرد، حافظه و دیگری زمان اجرای الگوریتم است. الگوریتمی همواره بهتر است که حافظه و زمان اجرای کمتری را داشته باشد، در این درس همواره زمان اجرای یک الگوریتم است که برای ما اهمیت دارد

۱.۱. ارزیابی مرتبه اجرایی یک الگوریتم (تابع زمانی)

مثال ۱.۱ کد رو به رو را از نظر مرتبه زمانی اجرا آنالیز کنید:

```
1 (1) int i = 0;  
2 (2) for(i=0; i<n; i++)  
3 (3)     x++;
```

پاسخ ۱.۱ در گام نخست جدول زمان و تکرار کد بالا را مینویسیم:

خط شماره	اجرا زمان	تکرار تعداد
(۱)	C_1	1
(۲)	C_2	$n + 1$
(۳)	C_3	n

در قطعه کد بالا عملیات های متفاوتی از جمله انتصاب، مقایسه ۷ تکرار و.. صورت گرفته. که هر کدام زمان های متفاوتی را برای اجرا شدن نیاز دارند.

با توجه به جدول فوق فرمول محاسبه مرتبه اجرایی یک الگوریتم یا تابع زمانی آن به صورت زیر خواهد بود:

$$T(n) = C_1 \times 1 + C_2 \times n + C_3 \times n$$

حال با فرض C بیشترین مقدار برای C_1 و C_2 و C_3 باشد داریم:

$$T(n) = C + Cn + C + Cn \rightarrow T(n) = 2C + 2Cn \rightarrow T(n) = C(2n + 2)$$

مثال ۲.۱ تابع زمانی الگوریتم روبه رو را بدست آورید:

```
1 int i = 0;
2 for(i=0;i<n;i++)
3     for(i=0;i<n;i++)
4         x++;
```

پاسخ ۲.۱ در گام نخست جدول زمان و تکرار کد بالا را مینویسیم:

خط شماره	اجرا زمان	تکرار تعداد
(۱)	C_1	1
(۲)	C_2	$n + 1$
(۳)	C_3	$n \times (n + 1)$
(۴)	C_4	n^2

$$T(n) = C_1 \times 1 + C_2 \times (n + 1) + C_3 \times n \times (n + 1) + C_4 \times n^2$$

حال با فرض اینکه C بیشترین مقدار زمانی است داریم:

$$\begin{aligned} T(n) &= C \times 1 + C \times (n + 1) + C \times n \times (n + 1) + C_4 \times n^2 \\ \rightarrow T(n) &= C + Cn + C + Cn^2 + Cn + Cn^2 \\ \rightarrow T(n) &= C(2n^2 + 2n + 2) \end{aligned}$$

مثال ۳.۱ تابع زمانی الگوریتم روبه رو را بدست آورید:

```
1 tni fact(tni n){
(1) tni fact = 1;
(2) rof(tni i=0;i<=n;i++)
(3)     fact*=i;
(4) nruter fact;}
```

پاسخ ۳.۱ در گام نخست جدول زمان و تکرار کد بالا را مینویسیم:

خط شماره	اجرا زمان	تکرار تعداد
(۱)	C_1	1
(۲)	C_2	n
(۳)	C_3	$(n-1)(n+1)$
(۴)	C_4	1

$$T(n) = C_1 \times 1 + C_2 \times n + C_3 \times (n-1) + C_4 \times 1$$

حال با فرض اینکه C بیشترین مقدار زمانی است داریم:

$$T(n) = C + Cn + Cn - C + C \rightarrow T(n) = C(2n + 1)$$

مثال ۴.۱ تابع زمانی الگوریتم روبه رو را بدست آورید:

```

do
    add(a, b, c, m, n){
(1)   for(i=0; i<=n; i++){
(2)       for(j=0; j<m; j++){
(3)           o[i][j] = a[i][j] + b[i][j];}
}
}

```

پاسخ ۴.۱ در گام نخست جدول زمان و تکرار کد بالا را مینویسیم:

خط شماره	اجرا زمان	تکرار تعداد
(۱)	C_1	$n+1$
(۲)	C_2	$n(m+1)$
(۳)	C_3	$n \times m$

$$T(n) = C_1 \times (n+1) + C_2 \times n(m+1) + C_3 \times n(m)$$

حال با فرض اینکه C بیشترین مقدار زمانی است داریم:

$$T(n) = C(n+1) + Cn(m+1) + Cnm \rightarrow T(n) = C(2nm + 2n + 1)$$

۲.۱ BigOmega Ω

گوئیم $T(n) \in \Omega(f(n))$ اگر و فقط اگر تابع صحیح C و n_0 وجود داشته باشد که به ازای همه مقادیر $n \geq n_0$ داشته باشیم

$$T(n) \geq Cf(n)$$

تعریف بالا را به صورت زیر نیز میتوان ارایه داد:

$$T(n) \in \Omega(f(n)) \leftrightarrow \exists C, n_0 \geq 0 \quad \forall n \geq n_0 \quad Cf(n) \leq T(n)$$

تعریف فوق یک کران پایین برای زمان اجرا برای $T(n)$ میدهد. بنابر این میتوان گفت که عبارت $\Omega(f(n))$ بهترین حالت اجرای یک الگوریتم است.

نکته در هنگام محاسبه $\Omega f(n)$ شرط زیر همواره برقرار است:

$$T(n) = a_m n^m + a_{m-1} n^{m-1} + \dots + a_1 n^1 + a_0 \rightarrow T(n) \in \Omega(f(n))$$

مثال ۵.۱ زمان اجرای یک الگوریتم به صورت زیر است، مطلوب است محاسبه $\Omega f(n)$:

$$T(n) = an^2 + bn + c$$

پاسخ ۵.۱ در مرحله نخست با توجه به نکته بالا و تابع موجود یک نامعادله شکل میدهیم و به دنبال n_0 و C هستیم که در عبارت زیر برقرار باشد.

$$an^2 + bn + c \geq an^2 + b + c \geq an^2$$

با توجه به عبارت بالا کمترین مقدار n_0 و C به صورت زیر است:

$$C = a \quad \text{and} \quad n_0 = 1$$

لذا رابطه زیر برای تابع برقرار است:

$$T(n) \in \Omega f(n^2)$$

مثال ۶.۱ زمان اجرای یک الگوریتم به صورت زیر است، مطلوب است محاسبه $\Omega f(n)$:

$$T(n) = n^4 + 5n^2$$

پاسخ ۶.۱ در مرحله نخست با توجه به نکته بالا و تابع موجود یک نامعادله شکل می‌دهیم و به دنبال n_0 و C هستیم که در عبارت زیر برقرار باشد.

$$n^4 + 5n^2 \geq n^4 + 5 \geq n^4$$

با توجه به عبارت بالا کمترین مقدار n_0 و C به صورت زیر است:

$$C = 1 \quad \text{and} \quad n_0 = 1$$

لذا رابطه زیر برای تابع برقرار است:

$$T(n) \in \Omega f(n^4)$$

مثال ۷.۱ زمان اجرای $T(n)$ مربوط به تعدادی الگوریتم موجود است. درستی عبارات زیر را ثابت کنید:

$$(1) T(n) = 6n + 4 \in \Omega(n)$$

$$(2) T(n) = 6n + 4 \notin \Omega(n)$$

$$(3) T(n) = 6n + 4 \in \Omega(n)$$

پاسخ ۷.۱ :

(۱) با توجه به تابع موجود یک نامعادله شکل می‌دهیم:

$$T(n) = 6n + 4 \geq 6n$$

با توجه به نامعادله فوق متوجه خواهیم شد که به ازای مقادیر $n_0 = 1$ and $C = 6$ رابطه $T(n) \in \Omega f(n)$ همواره برقرار خواهد بود.

(۲) با توجه به معادله موجود ، هیچ C و n_0 ای موجود نیست که در نامعادله زیر صدق کند:

$$T(n) = 3n + 2 \geq Cn^2$$

بنابر این رابطه زیر همواره درست خواهد بود:

$$3n + 2 \notin \Omega(n^2)$$

(۳) با توجه به تابع برای محاسبه $\Omega f(n)$ و اثبات رابطه یک نامعادله تشکیل می‌دهیم:

$$T(n) = 5^n + n^2 \geq 5^n \geq 2^n$$

نامعادله فوق به ازای مقادیر $n_0 = 1$ و $C = 1$ برای شرط $T(n) \in \Omega(2^n)$ برقرار است.

مثال ۸.۱ نشان دهید رابطه زیر برقرار است:

$$\frac{n(n-1)}{2} \in \Omega(n^2)$$

پاسخ ۸.۱ با توجه به تعریف $\Omega(f(n))$ داریم :

$$\frac{n(n-1)}{2} \geq cn^2$$

حال باید نشان دهیم به ازای یک C و n_0 رابطه برقرار است:

$$\xrightarrow{C=\frac{1}{4}} \frac{n^2 - n}{2} \geq \frac{n^2}{4} \rightarrow n_0 = 2$$

۳.۱ $BigO$ و O

میگوییم $T(n) \in O(f(n))$ اگر و فقط اگر ثابت C و ثابت صحیح n_0 وجود داشته باشد که برای همه مقادیر $n \geq n_0$ داشته باشیم $T(n) \leq Cf(n)$ در واقع $BigO$ کران بالای ما در زمان اجرای یک الگوریتم است، به عبارتی بالاترین زمان اجرایی الگوریتم. تعریف فوق را میتوان به صورت زیر نیز نمایش داد:

$$T(n) \in O(f(n)) \leftrightarrow \exists C, n_0 > 0 \quad \forall n \geq n_0 \quad T(n) \leq Cf(n)$$

مثال ۹.۱ با مطلوب است محاسبه O با توجه به تابع $T(n)$ ؟

$$T(n) = C(2n + 2)$$

پاسخ ۹.۱ در گام نخست باید برای C ضربی بیابیم که در نامعادله زیر صدق کند:

$$\begin{aligned} \xrightarrow{C=1} T(n) &= 2n + 2 \\ T(n) &= 2n + 2 \leq 3n \end{aligned}$$

متوجه خواهیم شد برای تابع $T(n)$ باقرار دادن $C = 1$ به یک فرم استاندارد برای این تابع خواهیم رسید و با تشکیل یک نامعادله به محاسبه $BigO$ این تابع میپردازیم، با توجه به نامعادله فوق داریم:

$$\text{for } f(n) = 3n \text{ we have : } n_0 = 2 \text{ and } C = 3$$

$$T(n) \in O(n)$$

مثال ۱۰.۱ مطلوب است محاسبه $BigO$ تابع زیر:

$$T(n) = 2n^2 + 4n$$

پاسخ ۱۰.۱

$$2n^2 + 4n \leq 3n^2 \rightarrow c = 3, f(n) = n^2 \text{ and } n_0 = 4$$

$$T(n) \in O(n^2)$$

نکته در هنگام محاسبه $O(f(n))$ شرط زیر همواره برقرار است

$$T(n) = a_m n^m + a_{m-1} n^{m-1} + \dots + a_1 n^1 + a_0 \rightarrow T(n) \in O(f(n))$$

مثال ۱۱.۱ مطلوب است محاسبه $BigO$ تابع زیر:

$$T(n) = 3n^3 + 3n$$

پاسخ ۱۱.۱

$$3n^3 + 3n \leq 4n^3 \rightarrow c = 4, f(n) = n^3 \text{ and } n_0 = 2$$

$$T(n) \in O(n^3)$$

مثال ۱۲.۱ نشان دهید رابطه زیر برقرار است:

$$n^2 + 10n \in O(n^2)$$

پاسخ ۱۲.۱ با توجه به تعریف $O(f(n))$ داریم:

$$n^2 + 10n \leq Cn^2$$

حال باید نشان دهیم به ازای یک C و n_0 رابطه برقرار است:

$$\xrightarrow{C=2} n^2 + 10n \leq 2n^2 \rightarrow n_0 = 10$$

مثال ۱۳.۱ نشان دهید رابطه زیر برقرار است:

$$\frac{n(n-1)}{2} \in O(n^2)$$

پاسخ ۱۳.۱ با توجه به تعریف $O(f(n))$ داریم:

$$\frac{n(n-1)}{2} \leq cn^2$$

حال باید نشان دهیم به ازای یک C و n_0 رابطه برقرار است:

$$\xrightarrow{C=\frac{1}{2}} n^2 + 10n \leq 2n^2 \rightarrow n_0 = 10$$

۴.۱ BigTheta Θ

تا کنون یک کران پایین و بالا برای تابع زمانی اجرای یک الگوریتم توسط نماد های $\mathbf{O}$ و $\mathbf{\Omega}$ ارائه دادیم. حال نماد Θ به صورت زیر تعریف میشود:

گوییم $T(n) \in \Theta f(n)$ اگر و فقط اگر ثابتهای C_1 و C_2 و ثابت صحیح n_0 وجود داشته باشد به طوریکه برای همه مقادیر $n \geq n_0$ داشته باشیم:

$$C_1 f(n) \leq T(n) \leq C_2 f(n)$$

همچنین تعریف بالا را میتوان به صورت زیر نیز ارائه داد:

$$\exists C_1, C_2, n_0 > 0 \quad \forall n \geq n_0 \quad C_1 f(n) \leq T(n) \leq C_2 f(n) \leftrightarrow T(n) \in \Theta$$

نکته در هنگام محاسبه $\Theta f(n)$ شرط زیر همواره برقرار است:

$$T(n) = a_m n^m + a_{m-1} n^{m-1} + \dots + a_1 n^1 + a_0 \rightarrow T(n) \in \Theta(f(n))$$

مثال ۱۴.۱ اگر تابع $T(n)$ به صورت روبه رو باشد، Θ را بدست آورید :

$$T(n) = \frac{1}{2}n^2 - 3n$$

پاسخ ۱۴.۱ نشان میدهیم که $T(n) \in \Theta(n^2)$ توجه به تعریف بالا داریم:

$$\exists C_1, C_2, n_0 > 0 \quad \forall n \geq n_0 \quad C_1 n^2 \leq \frac{1}{2}n^2 - 3n \leq C_2 n^2$$

حل بعد رابطه فوق را به n^2 تقسیم میکنیم:

$$C_1 \leq \frac{1}{2} - \frac{3}{n} \leq C_2$$

با توجه به عبارت بالا، طرف راست، به ازای $\frac{1}{2} \leq C_2$ و $n \geq 1$ برقرار است. به همین ترتیب برای طرف چپ عبارت بالا به ازای $n \geq 7$ و $C_1 \leq \frac{1}{4}$ حاصل میشود. بنابر این به ازای مقادیر زیر عبارت $T(n) \in \Theta(n^2)$ برقرار خواهد بود:

$$n_0 \geq 7, \quad C_2 = \frac{1}{2} \quad \text{and} \quad C_1 = \frac{1}{4}$$

مثال ۱۵.۱ فرض کنید تابع $T(n) = 7n^3$ باشد. آیا $T(n) \in \Theta(n^2)$ میتواند باشد یا خیر؟

پاسخ ۱۵.۱ طبق تعریف Θ باید داشته باشیم:

$$\exists \quad C_1, C_2, n_0 > 0 \quad \forall n \geq n_0 \quad C_1 n^2 \leq 7n^3 \leq C_2 n^2$$

طرف چپ بالا همیشه برقرار است اما طرف راست رابطه بالا در صورتی برقرار است که $n \leq \frac{C_2}{7}$ باشد و این با تعریف Θ که در آن رابطه برای هر n بزرگتر از n_0 برقرار است منافات دارد. لذا $T(n)$ نمیتواند متعلق به $\Theta(n^2)$ باشد

مثال ۱۶.۱ نشان دهید که $\frac{n(n-1)}{2} \in \Theta(n^2)$

پاسخ ۱۶.۱ با توجه به مثال های قبل دیدیم که $\frac{n(n-1)}{2} \in (n^2)$ و $\frac{n(n-1)}{2} \in \Omega(n^2)$ بود. در نتیجه بنابه تعریف Θ داریم:

$$\frac{n(n-1)}{2} \in \Theta(n^2)$$

که این عبارت برای تابع $T(n)$ درست و برقرار است.

مثال ۱۷.۱ زمان اجرای $T(n)$ مربوط به تعداد الگوریتم موجود است، درستی عبارات زیر را اثبات کنید.

$$(1) T(n) = 6n + 4 \in \Theta(n)$$

$$(2) T(n) = 3n + 6 \notin \Theta(n^2)$$

پاسخ ۱۷.۱ برای عبارت (۱) طبق تعریف Θ باید داشته باشیم:

$$\exists \quad C_1, C_2, n_0 > 0 \quad \forall n \geq n_0 \quad C_1 n \leq 6n + 4 \leq C_2 n$$

با توجه به رابطه بالا، طرف راست به ازای مقادیر $C_2 = 7$ و $n_0 = 2$ برقرار میباشد و طرف چپ رابطه به ازای $C_1 = 6$ و $n_0 = 1$ برقرار خواهد بود.

لذا به ازای مقادیر زیر رابطه $T(n) \in \Theta(n)$ برقرار خواهد بود.

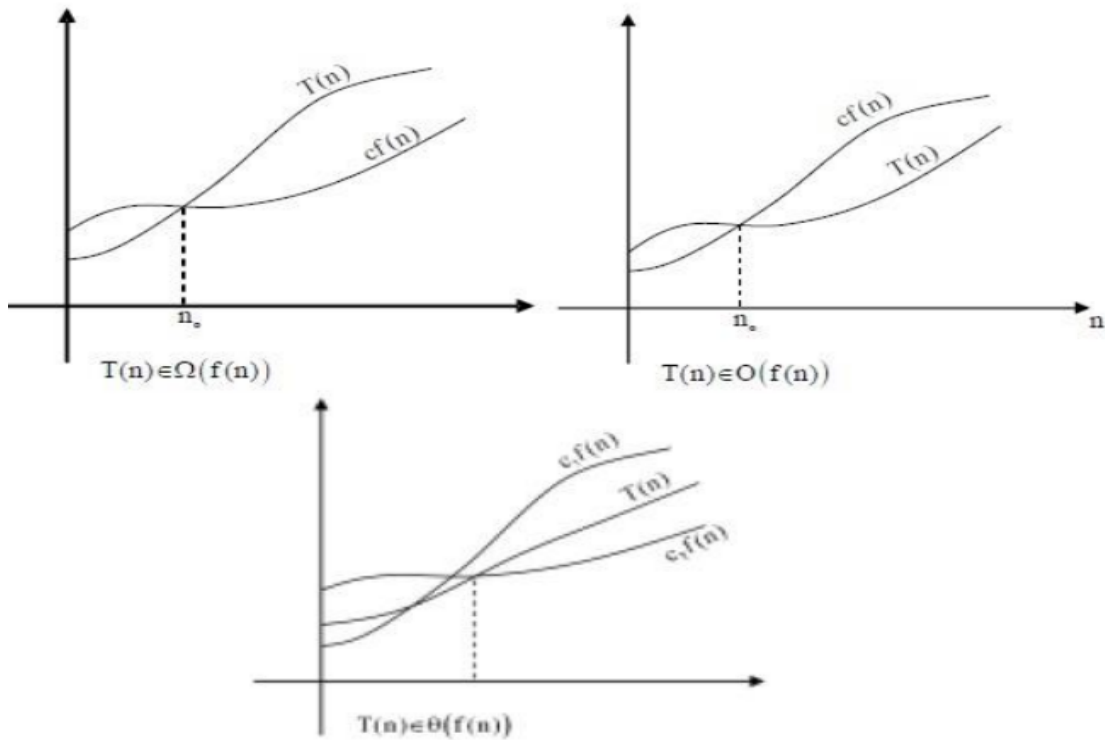
$$n_0 \geq 2, \quad C_2 = 7 \quad \text{and} \quad C_1 = 6$$

برای عبارت (۲) طبق تعریف Θ باید داشته باشیم:

$$\exists \quad C_1, C_2, n_0 > 0 \quad \forall n \geq n_0 \quad C_1 n^2 \leq 3n + 6 \leq C_2 n^2$$

طرف راست رابطه بالا همیشه برقرار است اما طرف چپ رابطه بالا برای هر n برقرار نیست و این با تعریف Θ که در آن برای هر n بزرگتر از n_0 منافات دارد، لذا نمیتوان تابع $T(n)$ متعلق به $\Theta(n^2)$ باشد.

شکل های زیر نماد های Θ ، Ω ، O را با استفاده از نمودار نشان میدهد. با توجه به تعریف نماد های بالا نمودار را ترسیم میکنیم، در این نمودار ها $T(n)$ تابع زمانی و $F(n)$ پیچیدگی زمانی تابع $T(n)$ میباشد.



۲ آرایه ها

آرایه ها Array مجموعه ای از داده های هم نوع که تحت یک نام معرفی می شوند و برای دسترسی به هر عنصر آن باید از اندیس استفاده کرد.

هر محتوایی که در آرایه ذخیره شود، همه تحت یک نام ذخیره شده و برای دسترسی به هر یک باید از اندیس مربوط به آن خانه استفاده کرد.

اندیس یا index: هر یک از عناصر موجود در یک آرایه یک اندیس یا شماره منحصر به فرد خود را دارند که برای دسترسی مستقیم به هر عنصر درون یک آرایه میتوان از اندیس مربوط به آن خانه آرایه استفاده کرد. بسته به نوع و کارکرد هر زبان برنامه نویسی، اندیس شروع یک آرایه میتواند از ۱ یا ۰ شروع شود.

در این جزوه به دلیل استفاده از شبه کدهای زبان سی، اندیس شروع یک آرایه را ۰ در نظر میگیریم.

۱.۲ انواع آرایه ها

آرایه ها بسته به تعداد سطر و ستون ای که دارند به سه نوع زیر تقسیم بندی میشوند:

۱. آرایه های یک بعدی:

آرایه های یک بعدی دارای یک سطر بوده و عناصر به صورت ستونی در هر خانه با اندیس مربوطه قرار میگیرند

۲. آرایه های دو بعدی:

آرایه های این دسته دارای چندین سطر و ستون بوده و عناصر به صورت سطری و ستونی در خانه های آرایه قرار میگیرند

۳. آرایه های سه بعدی:

این آرایه ها علاوه بر داشتن چندین سطر و ستون دارای لایه های متعدد نیز میباشند و عناصر برای قرار گیری در خانه این نوع آرایه ها علاوه بر محل قرار گیری سطری و ستونی خود باید در لایه مربوطه نیز قرار گیرند.

نحوه تعریف یک آرایه یک بعدی

```
1 type name[num];  
2 int a[10];
```

و برای دسترسی به عنصر درون آرایه یک بعدی:

```
1 int a[10];  
2 x = a[8];
```

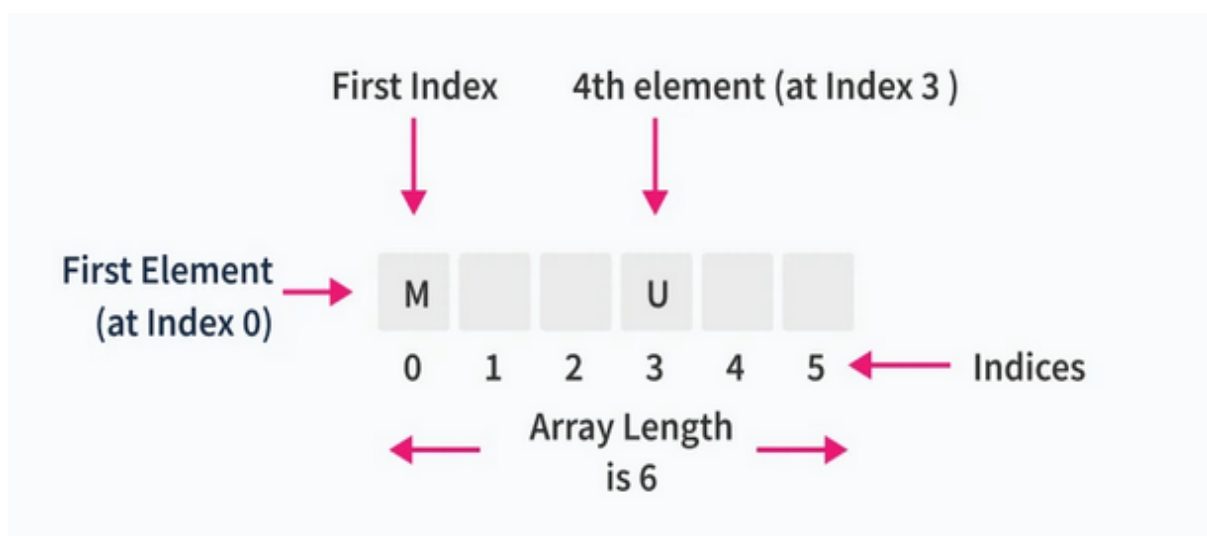
نکته:

کوچکترین مقدار اندیس یک آرایه را حد پایین آن آرایه گفته و آنرا Lower می نامند و با L آنرا نمایش میدهند. بزرگترین مقدار اندیس یک آرایه را حد بالای آن آرایه گفته و آنرا Upper می نامند و با U آنرا نمایش میدهند.

فرمول محاسبه تعداد عناصر یک آرایه یک بعدی :

$$Upper - Lower + 1$$

شکل یک آرایه یک بعدی دارای ۶ عنصر به صورت زیر میباشد:



اطلاعاتی که در آرایه یک بعدی ذخیره میشوند، در حافظه پشت سر هم قرار میگیرند ، این نحوه ذخیره سازی باعث ایجاد سرعت در دسترسی به عناصر آرایه میشود.

نحوه تعریف یک آرایه دو بعدی یا ماتریس

براکت اول برای تعداد سطر ها و براکت دوم برای تعداد ستون های آرایه دو بعدی خواهد بود.

```
1 type name[i][j];  
2 int array[4][3];
```

و برای دسترسی به عنصر درون آرایه دو بعدی:

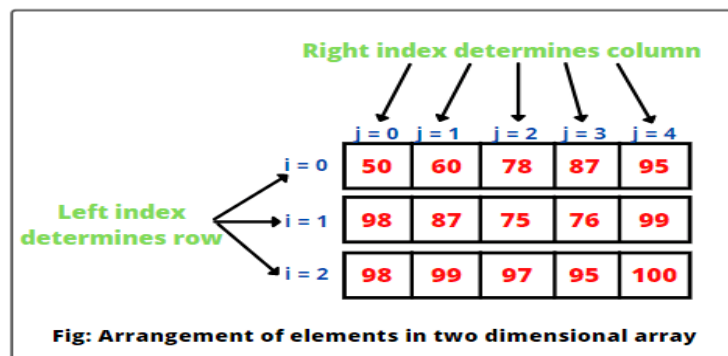
```
1 int a[4][3];  
2 x = a[2][1];
```


فرمول محاسبه تعداد عناصر یک آرایه دو بعدی :

تعداد سطر ها $\times$ تعداد ستون ها

در آرایه دو بعدی عناصر به صورت سطری و ستونی در حافظه ذخیره میشوند.

شکل یک آرایه دو بعدی (ماتریس) :



نحوه تعریف یک آرایه سه بعدی

براکت اول به معنای تعداد آرایه های دوبعدی (لایه ها)

براکت دوم اشاره به تعداد سطر آرایه های دوبعدی دارد.

براکت سوم اشاره به تعداد ستون های آرایه دوبعدی دارد.

```
1 type name[i][j][k];  
2 int array[3][4][5];  
3
```

و برای دسترسی به عنصر درون آرایه سه بعدی:

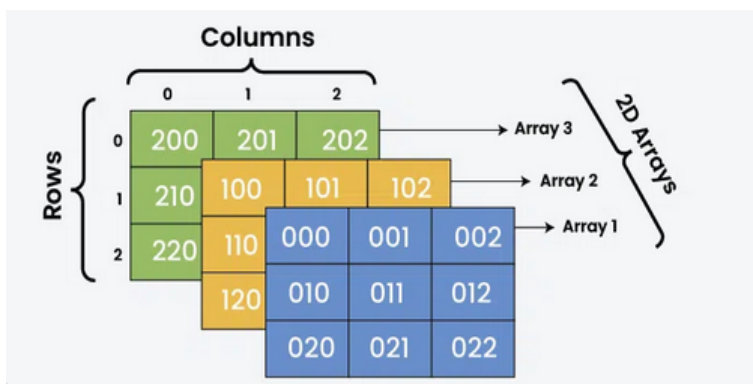
```
1 int a[3][4][5];  
2 x = a[3][2][4];
```

فرمول محاسبه تعداد عناصر یک آرایه دو بعدی :

تعداد لایه ها $\times$ تعداد سطر ها $\times$ تعداد ستون ها

در آرایه های ۳ بعدی ما به تعداد براکت اول ، لایه داریم و در هر لایه یک ماتریس وجود دارد به اندازه براکت های دوم و سوم. لازم به ذکر است که در هر لایه آرایه دو بعدی از نظر اندیس تکرار خواهد شد و مبنای جداسازی ما براساس لایه ها خواهد بود.

شکل یک آرایه سه بعدی:



۲.۲ نحوه ذخیره سازی عناصر در آرایه

عناصر آرایه در حافظه پشت سر هم ذخیره می شوند و به همین علت دسترسی به آنها سریع است. با فرض اینکه عنصر اول آرایه در آدرس α حافظه ذخیره شود و هر عنصر آرایه به اندازه ω بایت فضا اشغال نماید. با فرض اینکه عناصر در حافظه به صورت سطر به سطر ذخیره شده اند، محل هر عنصر آرایه در حافظه به کمک روابط زیر محاسبه می شود:

نکته: در کامپایلر هایی مثل کامپایلر C عناصر زمانیکه در حافظه ریخته می شوند سطر به سطر ریخته میشوند، کامپایلر هایی داریم که ذخیره سازی را به صورت ستونی انجام می دهند.

آرایه های یک بعدی

$X[L \dots U]$ یک آرایه از اندیس L تا U است ، برای بدست آوردن محل ذخیره سازی عنصر i در حافظه داریم:
لازم به ذکر است در زبان C اندیس شروع $L = 0$ میباشد.

$$Loc(x[i]) : \alpha + (i - l) \times \omega$$

آرایه های دو بعدی

$X[L_1 \dots U_2, L_2 \dots U_2]$ یک آرایه از L_1 تا U_1 اندیس های سطری و L_2 و U_2 اندیس های ستونی ماست برای بدست آوردن محل ذخیره سازی عنصر I_2 در حافظه داریم:
به صورت سطری:

$$Loc(x[i, j]) : \alpha + [(i - L_1) \times (U_2 - L_2 + 1) + (j - L_2)] \times \omega$$

به صورت ستونی:

$$Loc(x[i, j]) : \alpha + [(j - L_2) \times (U_1 - L_1 + 1) + (i - L_1)] \times \omega$$

آرایه های سه بعدی

$X[L_1 \dots U_2, L_2 \dots U_2, L_3 \dots U_3]$ اگر آرایه ۳ بعدی باشد برای یافتن آدرس یک عنصر در حافظه از فرمول زیر استفاده میکنیم:

$$Loc(x[i, j, k]) : \alpha + [(i - L_1) \times (U_2 - L_2 + 1) \times (U_3 - L_3 + 1) + (j - L_2) \times (U_3 - L_3 + 1) + (k - L_3)] \times \omega$$

نکته :

آدرس سطری و ستونی معمولا حاصل یکسانی ندارند به دلیل نحوه ذخیره سازی عناصر در حافظه . در روش سطری عناصر هر سطر به صورت پشت سر هم در حافظه ذخیره میشوند یعنی اول تمام عناصر سطر اول سپس همه عناصر سطر دوم و به همین طریق . در روش ستونی عناصر هر ستون به صورت پشت سر هم ذخیره می شوند به عبارتی ابتدا تمام عناصر ستون اول سپس تمام عناصر ستون دوم و به همین طریق

مثال ۱.۲ در آرایه دو بعدی زیر آدرس سطری و ستونی خانه خواسته شده را بدست آورید:
(شروع اندیس آرایه از ۱ میاشد)

$$\omega = 1 \text{ and } \alpha = 100$$

۱	۲	۷	۲۰	۲۱
۱۷	۱۹	۶	۵	۲۰
۱۸	۰	۱	۲	۴

پاسخ ۱.۲ با توجه به فرمول محاسبه آدرس های دوبعدی داریم:

برای عنصر ۲۰:

سطری:

$$(1 - 1)(5 - 1 + 1) + (4 - 1)(1) + 100 = 103$$

ستونی:

$$(4 - 1)(3 - 1 + 1) + (1 - 1)(1) + 100 = 109$$

برای عنصر ۶:

سطری:

$$(2 - 1)(5 - 1 + 1) + (3 - 1)(1) + 100 = 107$$

ستونی:

$$(3 - 1)(3 - 1 + 1) + (2 - 1)(1) + 100 = 106$$

مثال ۲.۲ آدرس سطری عنصر $A[1][2]$ را در آرایه $A[3][4]$ محاسبه کنید.
با فرض اینکه آرایه از نوع داده ای ۸ بایتی است و آدرس شروع آرایه ۰ باشد.

۰	۱	۲	۳
۴	۵	۶	۷
۸	۹	۱۰	۱۱

پاسخ ۲.۲ با توجه به مقادیر موجود $\alpha = 10$ و $\omega = 8$ و داشتن محل عنصر کافیت در فرمول جاگذاری و حاصل را بیابیم.

$$[(1 - 0)(4 - 1 + 1) + (2 - 0)](8) + 10 = 58$$

مثال ۳.۲ آدرس عنصر $A[3][4][2]$ را در آرایه $A[20][10][5]$ محاسبه کنید.
با فرض اینکه آرایه از نوع داده ای ۲ بایتی است و آدرس شروع آرایه ۰ باشد.

پاسخ ۳.۲ کافیت مقادیر موجود را در فرمول محاسبه آرایه سه بعدی درج و محاسبات را انجام دهیم.

$$[(3 - 0) \times (10) \times (5) + (4 - 0) \times 5 + (2 - 0)](2) + 0 = 344$$

۳.۲ عملیات روی آرایه ها

انواع عملیات بر روی آرایه ها:

۱. جستجو در آرایه

(آ) جستجو خطی Linear Search

(ب) جستجو دودویی Binary Search

۲. درج

۳. حذف

جستجو خطی Linear search

در این شیوه جستجو ، در آرایه به جستجو عنصر خواصی میپردازیم. روش کار این نوع جستجو این است که به ترتیب از خانه اول حرکت کرده تا خانه آخر آرایه و بررسی میکنیم که در هر خانه ، آرایه عنصر مدنظر ما وجود دارد یا نه. تابع زیر مقدار item را در آرایه n عنصری a ، به روش مقایسه با تک تک عناصر ، آرایه را جستجو می نماید و در صورت پیدا کردن ، اندیس خانه حاوی item و در غیر اینصورت عدد 1- را برمیگرداند. لازم به ذکر است که در یک جستجو ناموفق نیاز به $n + 1$ عمل مقایسه داریم که در نتیجه تابع زمانی این جستجو برابر با $O(n)$ خواهد بود.

```
1 int linear_search(a[], n ,item)
2 {
3     int i;
4     for(i=0;i<n;i++){
5         if(a[i] == item)
6             return(i);
7     }
8     return(-1);
9 }
```

جستجو دودویی Binary Search

با فرض اینکه آرایه به طور صعودی مرتب شده باشد، عضو مورد جستجو را با عنصر وسط آرایه مقایسه میشود، در صورت برابر بودن پیدا شده است.

در غیر اینصورت اگر از عنصر وسط آرایه بزرگتر باشد، مقایسه به طور بازگشتی در نیمه بالای آرایه انجام میشود و در صورت کوچکتر بودن از عنصر وسط، مقایسه به طور بازگشتی در نیمه پایین آرایه انجام می شود.

در صورتیکه آرایه مرتب نباشد، عمل جستجو دودویی بر روی آن ممکن نیست.

مثال ۴.۲ با کمک روش جستجو دودویی عدد ۱۲ را در آرایه مرتب زیر پیدا کنید.

۵	۹	۱۲	۲۰	۳۵	۵۰	۸۲	۸۸	۹۷
---	---	----	----	----	----	----	----	----

در صورت استفاده از جستجو خطی با ۳ مقایسه به مقدار مورد نظر خواهیم رسید.

پاسخ ۴.۲ برای جستجو دودویی در مرحله نخست باید عنصر وسط را پیدا کنیم. عنصر وسط این آرایه ۳۵ می باشد. بعد از پیدا شدن عدد وسط آرایه، عدد ۱۲ را با ۳۵ که عنصر وسط آرایه بود مقایسه میکنیم. به دلیل اینکه عدد ۱۲ از ۳۵ کوچک تر است لذا طرف سمت راست عدد ۳۵ و همچنین خود ۳۵ را از بازه جستجو حذف میکنیم و به آرایه زیر میرسیم.

۵	۹	۱۲	۲۰
---	---	----	----

در مرحله بعد عدد ۱۲ با نیمه پایین عنصر وسط یعنی ۹ مقایسه میشود، به دلیل اینکه ۱۲ از ۹ بزرگتر است لذا طرف سمت چپ عدد ۹ به همراه خود ۹ از بازه جستجو حذف شده و به آرایه زیر خواهیم رسید.

۱۲	۲۰
----	----

در نهایت فقط به ۲ خانه رسیدیم، با همان روش قبلی میانگین و مقدار با خانه بدست آمده چک میشود. به دلیل برابری عدد ۱۲ با عنصر ۱۲ در آرایه لذا عمل سرچ به اتمام میرسد.

جستجو دودویی به دلیل مرتبه پایین تر به نسبت جستجو خطی، جستجو بهینه تری می باشد، کارایی جستجو دودویی در زمان نیست که آرایه با تعداد عناصر زیادی داشته باشیم و جستجو خطی برای آرایه با عناصر نسبتاً کم میباشد.

دو تابع زیر مقدار `item` را در آرایه `n` عنصری `a`، به روش دودویی جستجو می نمایند و در صورت پیدا کردن، اندیس خانه حاوی `item` و در غیر اینصورت عدد `-1` را برمیگردانند (در ابتدا $L = 0$ و $U = n - 1$ است).

جستجوی دودویی به روش غیر بازگشتی

```
1 int binary_search(int a[], item, L, U){
2     while(l <= U){
3         int mid = (U - L)/2;
4         if(a[mid] == item)
5             return mid;
6         if(a[mid] > item)
7             U = mid - 1;
8         else
9             L = mid + 1;}
10    return(-1);
11 }
```

جستجوی دودویی به روش بازگشتی

```
1 int binary_search(int a[], item, L, U){
2     if(l <= U){
3         mid = (L + u)/2;
4         if(item < a[mid])
5             binary_search(a,item,L,mid-1);
6         elseif(item > a[mid])
7             binary_search(a,item,mid+1,U);
8         else
9             return mid;
10    }
11    return(-1);
12 }
```

مثال ۵.۲ با روش جستجو خطی تحلیل کنید عدد ۶ در آرایه $int\ col[] = \{5, 3, 8, 6, 2\}$ وجود دارد یا خیر؟

پاسخ ۵.۲ یک جدول ایجاد کرده و رفتار تابع جستجو خطی برای این آرایه را بررسی میکنیم:

وضعیت شرط درون حلقه i	عنصر آرایه	شمارنده حلقه
ادامه حلقه	۲	۱
ادامه حلقه	۳	۲
ادامه حلقه	۸	۳
پیدا شدن مقدار مورد نظر خروج از حلقه	۶	۴

با پیدا شدن مقدار مورد نظر تابع اندیس مربوط به محل قرار گیری عنصر را برمیگرداند.

مثال ۶.۲ با روش جستجو دودویی تحلیل کنید عدد ۶ و ۱۰ در آرایه زیر وجود دارند یا خیر:

$$int\ A[] = \{2, 4, 6, 8, 10, 12, 14, 16, 18\}$$

پاسخ ۶.۲ برای یافتن عدد ۱۰ در آرایه فوق :

با توجه به اینکه شروع اندیس از ۰ میباشد ، اندیس بالای این آرایه برابر ۸ می باشد.

در مرحله نخست باید میانگین خانه این آرایه را محاسبه کرده و عدد ۱۰ را با آن مقایسه کنیم .

$$L = 0 \quad and \quad U = 8 \rightarrow mid = \frac{8 + 0}{2} = 4$$

$$A[mid] == 10 \quad is \quad True \rightarrow return \quad mid$$

لذا با پیدا شدن مقدار مورد نظر، جستجو دودویی به پایان رسید و تابع اندیس خانه ای که عنصر در آن بود را برمیگرداند.

برای یافتن عدد ۶ در آرایه فوق :

دوباره مقدار ۶ را با عنصر وسط آرایه چک میکنیم ، عدد ۶ از وسط آرایه کوچکتر است لذا از بازه جستجو عدد ۱۰ و اعداد سمت راست حذف میشوند.

$$6 < A[mid] \rightarrow U = mid - 1 \rightarrow U = 3 \quad and \quad L = 0$$

حال با تغییر بازه جستجو دوباره میانگین عنصر وسط بازه را محاسبه کرده و با مقدار مورد نظر چک میکنم.

$$mid = int(\frac{3 + 0}{2}) = 1 \rightarrow A[mid] < 6 \rightarrow L = mid + 1 = 2 \quad and \quad U = 3$$

در نهایت باز هم با کوچک تر شدن بازه جستجو، برای آخرین بار عمل میانگیری و مقایسه با ۶ را انجام میدهیم.

$$mid = int(\frac{3 + 2}{2}) = 2 \rightarrow A[mid] = 6 \rightarrow return \quad mid$$

لذا در اندیس شماره ۳ آرایه ، عنصر مدنظر ما وجود داشت و تابع آدرس اندیس مربوطه را بعد از یافتن برمی گرداند.

درج در آرایه insert

تابع insert مقدار item را در مکان k ام آرایه a اضافه میکند.

درج در آرایه به دلیل فرآیند insert، زمانبر است، این فرآیند بیان میکند که برای درج کردن مقداری در مکان k ام یک آرایه باید تمام عناصر در سمت راست آن خانه و خانه مد نظر همه یک اندیس به جلو بروند shift انجام شود تا بتوانیم عنصر را در مکان مورد نظر در آرایه درج کنیم، به همین دلیل در تابع پارامتری به نام m داریم که تعداد خانه های پر یک آرایه را نمایش میدهد.

زیر مقادیر به

```
1 binary_search(int a[], m, K, item){
2     for(i=m-1;i>=k;i--)
3         a[i+1] = a[i];
4     a[k] = item;
5 }
```

مثال ۷.۲ با توجه به مقادیر زیر، عمل درج در آرایه زیر را تحلیل کنید:

$$n = 7, \quad k = 0, \quad item = A \quad \text{and} \quad m = 5$$

B	C	D	E	F		
---	---	---	---	---	--	--

پاسخ ۷.۲ می خواهیم در خانه ای با اندیس ۰ مقدار A را قرار دهیم لذا به طریق زیر پیش میرویم:

در محله نخست F با اندیس ۴ را یک خانه به جلو میبریم و در اندیس ۵ قرار میدهیم

سپس همین کار را برای E انجام داده و از اندیس ۳ به ۴ شیف्ट میدهیمش.

این کار را تا زمانی که اندیس خانه مورد نظر خالی شود انجام میدهیم لذا به آرایه به فرم زیر تغییر میکند:

	B	C	D	E	F	
--	---	---	---	---	---	--

حال با توجه به خالی شدن اندیس مورد نظر آرایه item را درج کرده و شکل نهایی آرایه به صورت زیر خواهد بود:

A	B	C	D	E	F	
---	---	---	---	---	---	--

حذف از آرایه delete

تابع زیر kامین عنصر آرایه را حذف کرده و آنرا در متغیر x قرار میدهد. شرط حذف ($n \geq k$)
فرآیند حذف از یک آرایه با حذف آن عنصر از آرایه و سپس شیفت تمامی عناصر در آرایه به سمت چپ صورت میگیرد و بعد از خروج از حلقه آخرین خانه آرایه را نیز برابر ۰ قرار میدهیم.

```
1 delete(int a[], n, K, item){
2     int x = a[k]
3     for(i=k;i<n;i++)
4         a[i] = a[i+1];
5     a[i] = 0;
6     return(x);
7 }
```

۴.۲ ماتریس ها Matrix

به هر آرایه دو بعدی که دارای m سطر و n ستون باشد ماتریس میگویند، اما بحث ما برای پیاده سازی ماتریس نیست بلکه با ماتریس های روبه رو هستیم که در آن ها عناصر تکراری یا ۰ های زیادی وجود دارد.
برای ذخیره سازی این عناصر فضا را اشغال کرده اند و می خواهیم آن اعداد یا عناصری را ذخیره کنیم که ۰ یا تکراری نباشند. لذا بحث در مورد صرفه جویی در مصرف حافظه خواهد بود.

انواع ماتریس ها:

۱. ماتریس مربعی

۲. ماتریس مثلثی (بالا مثلثی و پایین مثلثی)

۳. ماتریس قطری (سه قطری، پنج قطری و..)

۴. ماتریس پراکنده (خلوت یا اسپارس)

ماتریس مربعی در این نوع ماتریس ها تعداد سطر و ستون با هم برابر ($n=m$) است به طور مثال ماتریس A یک ماتریس مربعی می باشد.

$$A = \begin{bmatrix} A & B & C \\ D & E & F \\ F & G & H \end{bmatrix}$$

ماتریس خلوت (اسپارس)

ماتریسی که دارای تعداد نسبتاً زیادی عنصر ۰ باشد را ماتریس اسپارس یا خلوت می نامند ، در این ماتریس برای کاهش حافظه مصرفی و زمان اجرا فقط عناصر غیر ۰ ماتریس ذخیره می شوند.
نحوه ذخیره سازی این ماتریس به صورت زیر است.

$$\begin{bmatrix} 0 & 0 & 2 & 0 \\ 0 & 5 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \xrightarrow{\text{change to}} \begin{bmatrix} 3 & 4 & 2 \\ 1 & 3 & 2 \\ 2 & 2 & 4 \end{bmatrix}$$

در ماتریس ۱ یک ماتریس اسپارس داریم که از کل درایه های موجود در این ماتریس فقط دو درایه دارای مقدار غیر ۰ میباشد. لذا آنرا تبدیل به ماتریس ۲ میکنیم:

در ماتریس ۲ داریم :

سطر اول اشاره دارد که ماتریس اصلی ما دارای ۳ سطر و ۴ ستون است و دو عنصر غیر صفر دارد.

به سطر اول این ماتریس که اطلاعاتی راجب به فرم اولیه ماتریس دارد header میگویند.

سطر دوم این ماتریس میگوید در سطر ۱ و ستون ۳، عدد ۲ قرار دارد.

سطر سوم این ماتریس بیان میکند در سطر ۲ و ستون ۲ این ماتریس عدد ۵ قرار دارد .

مثال ۸.۲ ماتریس زیر به ماتریسی با حافظه مصرفی و زمان اجرا کمتر تبدیل کنید.

$$\begin{bmatrix} 0 & 0 & 0 & 0 & 15 & 0 \\ 17 & 0 & 0 & 21 & 0 & 0 \\ 0 & 16 & 0 & 0 & 0 & 19 \\ 0 & 0 & 13 & 18 & 0 & 0 \\ 0 & 20 & 0 & 0 & 0 & 0 \end{bmatrix}$$

پاسخ ۸.۲ با توجه به شکل ماتریس متوجه خواهیم شد که ماتریس ما از نوع اسپارس بود و در نتیجه این تبدیل باید از نوع تبدیل ماتریس اسپارس باشد.

در مرحله نخست با توجه آرایه دیتا هایی که لازم داریم برای تبدیل ماتریس را جمع آوری میکنیم.

اندازه ماتریس 6×6

تعداد درایه هایی که صفر نمی باشند و بدست آوردن آدرس سطری و ستونی هر یک از این درایه ها.
در نهایت رسم این ماتریس.

$$\begin{bmatrix} 5 & 6 & 8 \\ 0 & 4 & 15 \\ 1 & 0 & 17 \\ 1 & 3 & 21 \\ 2 & 1 & 16 \\ 2 & 5 & 19 \\ 3 & 2 & 13 \\ 3 & 3 & 18 \\ 4 & 1 & 20 \end{bmatrix}$$

ماتریس مثلثی

در ماتریس مربعی اگر عناصر زیر قطر اصلی ۰ باشند به آن ماتریس بالا مثلثی میگوییم و اگر عناصر بالای قطر اصلی صفر باشد به ماتریس پایین مثلثی میگویند.
شکل زیر دو ماتریس پایین مثلثی و بالا مثلثی را نشان میدهد.

$$\begin{bmatrix} 1 & 0 & 0 \\ 2 & 2 & 0 \\ 8 & 4 & 6 \end{bmatrix} \quad \begin{bmatrix} 8 & 7 & 4 \\ 0 & 1 & 1 \\ 0 & 0 & 5 \end{bmatrix}$$

ماتریس بالا مثلثی ماتریس پایین مثلثی

عمل تبدیل ماتریس مثلثی :

در این تبدیل عناصر ماتریس مثلثی را سطر به سطر در آرایه یک بعدی ذخیره میکنیم، به عبارتی از یک ماتریس به آرایه یک بعدی میرسیم.

$$\begin{bmatrix} 2 & 0 & 0 \\ 3 & 6 & 0 \\ 5 & 4 & 1 \end{bmatrix} \xrightarrow{\text{change to}} \begin{array}{|c|c|c|c|c|c|} \hline a & a+1 & a+2 & a+3 & a+4 & a+5 \\ \hline 2 & 3 & 6 & 5 & 4 & 1 \\ \hline \end{array}$$

برای اینکه بتوان موقعیت عنصر را در آرایه یک بعدی بدست آوریم، به عبارتی عنصری که در ستون j و سطر i ماتریس مثلثی بوده در آدرس حافظه زیر در آرایه یک بعدی ذخیره میشود.
 α آدرس شروع حافظه می باشد.

$$\alpha + \frac{i(i-1)}{2} + j - 1$$

به طور مثال در ماتریس فوق عدد ۴ دارای $i=3$ و $j=2$ میباشد، موقعیت این عنصر در آرایه یک بعدی به صورت زیر است.

$$\alpha + \frac{3(2)}{2} + 1 = \alpha + 4$$

در نتیجه عدد ۴ در خانه $\alpha + 4$ آرایه یک بعدی قرار گرفته است.

ماتریس ۳ قطری

ماتریس قطری: هر ماتریس $n \times n$ که در آن L قطر که شامل قطر اصلی هم میباشد مخالف ۰ باشد. نمونه ای ماتریس قطری به صورت زیر است.

$$\begin{bmatrix} 1 & 0 & 0 \\ 9 & 5 & 0 \\ 0 & 2 & 3 \end{bmatrix}$$

ماتریس ۳ قطری یک ماتریس مربعی میباشد، که درایه های غیر ۰ آن روی قطر اصلی و بلافاصله بالا و پایین قطر اصلی ظاهر شوند. شکل ماتریس ۳ قطری به صورت زیر است:

$$\begin{bmatrix} 6 & 23 & 0 & 0 \\ 1 & 3 & 7 & 0 \\ 0 & 2 & 4 & 83 \\ 0 & 0 & 9 & 5 \end{bmatrix}$$

تعداد عناصر غیر ۰ این ماتریس از رابطه زیر بدست می آید.

$$3n - 2$$

همانند ماتریس مثلثی ذخیره سازی این ماتریس نیز در یک آرایه یک بعدی انجام میشود. از سطر اول شروع کرده و عناصر را به ترتیب در آرایه یک بعدی ذخیره میکنیم.

a	a+1	a+2	a+3	a+4	a+5	a+6	a+7	a+8	a+9
6	23	1	3	7	2	4	83	9	5

باید یک لینک برای عناصر ایجاد کنیم یعنی هر عنصر درون ماتریس، درون آرایه یک بعدی در چه وضعیتی قرار دارد، برای این منظور با فرض اینکه اولین عنصر در آدرس α باشد، رابطه زیر را داریم.

$$\alpha + 2i + j - 3$$

۳ زیر برنامه های بازگشتی

به طور کلی روش تحلیل الگوریتم ها به دو صورت زیر انجام میشود:

۱. ترتیبی

۲. بازگشتی

در این فصل به تحلیل الگوریتم های بازگشتی خواهیم پرداخت.

۱.۳ تابع بازگشتی recursive

تابعی که حداقل دارای یک دستور باشد که خود تابع را صدا بزند، این تابع به تعداد مراحل محدودی اجرا میشود و پس از آن متوقف میشود.

مثال هایی از توابع بازگشتی معروف:

□ فاکتوریل

□ مجموع اعداد ۱ تا n

□ توان

□ ترکیب

□ خارج قسمت تقسیم صحیح

□ فیبوناچی

□ هانوی

تابع بازگشتی فاکتوریل

تابع زیر فاکتوریل عدد n را محاسبه میکند.
در این تابع، تابع f در داخل خودش صدا زده شده است.

```

1 f(n){
2     if(n==1)
3         return 1;
4     else
5         return n*f(n-1);
6 }
7

```

$$f(n) = \begin{cases} 1 & n = 1 \\ n \times f(n-1) & n > 1 \end{cases}$$

مثال ۱.۳ فراخوانی تابع فاکتوریل با مقدار ۴ را تحلیل کنید.

$$\begin{aligned}
 f(4) &= 4 \times f(3) \xrightarrow{f(3)=6} f(4) = 4 \times 6 = 24 \\
 f(3) &= 3 \times f(2) \xrightarrow{f(2)=2} f(3) = 3 \times 2 = 6 \\
 f(2) &= 2 \times f(1) \xrightarrow{f(1)=1} f(2) = 2 \times 1 = 2 \\
 f(1) &= f(1) = 1
 \end{aligned}$$

پاسخ ۱.۳

مجموع اعداد ۱ تا n

تابع زیر جمع اعداد ۱ تا n را محاسبه میکند.

```

1 sum(n){
2     if(n==1)
3         return 1;
4     else
5         return n+sum(n-1);
6 }
7

```

$$sum(n) = \begin{cases} 1 & n = 1 \\ n + sum(n-1) & n > 1 \end{cases}$$

مثال ۲.۳ خروجی تابع sum برای $n=3$ را بدست آورید.

$$\begin{aligned} \text{sum}(3) &= 3 + \text{sum}(2) \xrightarrow{\text{sum}(2)=3} \text{sum}(3) = 3 + 3 = 6 \\ \text{sum}(2) &= 2 + \text{sum}(1) \xrightarrow{\text{sum}(1)=1} \text{sum}(2) = 2 + 1 = 3 \\ \text{sum}(1) &= \text{sum}(1) \end{aligned}$$

پاسخ ۲.۳

تابع توان

تابع زیر مقدار n^m را محاسبه میکند. (دو ورودی، ورودی ها صحیح و مثبت)

```

1 pow(n,m){
2     if(m==1)
3         return n;
4     else
5         return n*pow(n,m-1);
6 }
```

$$\text{pow}(n) = \begin{cases} n & m = 1 \\ n \times \text{pow}(n, m - 1) & m > 1 \end{cases}$$

مثال ۳.۳ تابع pow را به صورت pow(3,4) فراخوانی میکنیم، تابع را به صورت بازگشتی تحلیل کنید.

$$\begin{aligned} \text{pow}(3,4) &= 3 \times \text{pow}(3,3) \xrightarrow{\text{pow}(3,3)=27} \text{pow}(3,4) = 3 \times 27 = 81 \\ \text{pow}(3,3) &= 3 \times \text{pow}(3,2) \xrightarrow{\text{pow}(3,2)=9} \text{pow}(3,3) = 3 \times 9 = 27 \\ \text{pow}(3,2) &= 3 \times \text{pow}(3,1) \xrightarrow{\text{pow}(3,1)=3} \text{pow}(3,2) = 3 \times 3 = 9 \\ \text{pow}(3,1) &= 3 \end{aligned}$$

پاسخ ۳.۳

ترکیب

تابع com ترکیب m از n را محاسبه میکند. تابع آماری ترکیب به صورت زیر است.

$$(n, r) = \frac{n!}{r!(n-r)!}$$


```

1 com(n,m){
2     if((n==m) || (m==0))
3         return 1;
4     else
5         return com(n-1,m) + com(n-1,m-1);
6 }

```

$$com(n) = \begin{cases} \binom{n}{m} = 1 & \text{if } m = 0 \text{ or } m = n \\ \binom{n}{m} = \binom{n-1}{m} + \binom{n-1}{m-1} & \text{if } 0 < m < n \end{cases}$$

مثال ۴.۳ تابع com را به صورت pow(4,2) فراخوانی می کنیم، تابع را به صورت بازگشتی تحلیل کنید.

$$com(4,2) = com(3,2) + com(3,1) \xrightarrow[\com(3,2)=3]{\com(3,1)=3} com(4,2) = 3 + 3 = 6 \quad \text{پاسخ ۴.۳}$$

$$com(3,2) = com(2,2) + com(3,2) \xrightarrow[\com(2,2)=1]{\com(2,1)=2} com(3,1) = 1 + 2 = 3$$

$$com(3,1) = com(2,1) + com(2,0) \xrightarrow[\com(2,1)=2]{\com(1,1)=1} com(3,1) = 2 + 1 = 3$$

$$com(2,1) = com(1,1) + com(1,0) \xrightarrow[\com(1,0)=1]{\com(1,1)=1} com(2,1) = 1 + 1 = 2$$

خارج قسمت تقسیم صحیح

تابع زیر محاسبه خارج قسمت تقسیم صحیح a بر b را انجام میدهد.

```

1 div(a,b){
2     if(n==0)
3         return 0;
4     else
5         return div(a-b,b)+1;
6 }

```

$$div(n) = \begin{cases} 0 & a < b \\ div(a-b, b) + 1 & a \geq b \end{cases}$$

مثال ۵.۳ تحلیل فراخوانی تابع $\text{div}(11,3)$ به صورت بازگشتی.

$$\begin{aligned} \text{div}(11,3) &= \text{div}(8,3) + 1 \xrightarrow{\text{div}(8,3)=2} \text{div}(11,3) = 2 + 1 = 3 \\ \text{div}(8,3) &= \text{div}(5,3) + 1 \xrightarrow{\text{div}(5,3)=1} \text{div}(8,3) = 1 + 1 = 2 \\ \text{div}(5,3) &= \text{div}(2,3) + 1 \xrightarrow{\text{div}(2,3)=0} \text{div}(5,3) = 0 + 1 = 1 \end{aligned}$$

پاسخ ۵.۳

تابع بازگشتی فیبوناچی

محاسبه جمله n ام سری فیبوناچی

در سری فیبوناچی عدد بعدی از جمع دو عدد قبلی بدست خواهد آمد.

```

1 fib(n){
2     if((n==0) || (n==1))
3         return n;
4     else
5         return fib(n-1) + fib(n-2);
6 }
```

$$\text{div}(n) = \begin{cases} 0 & n = 0 \\ 1 & n = 1 \\ \text{fib}(n-1) + \text{fib}(n-2) & n \geq 2 \end{cases}$$

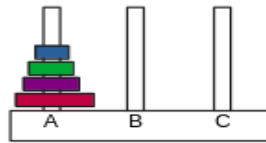
مثال ۶.۳ محاسبه جمله چهارم دنباله فیبوناچی به صورت بازگشتی.

$$\begin{aligned} \text{fib}(4) &= \text{fib}(3) + \text{fib}(2) \xrightarrow[\text{fib}(3)=2]{\text{fib}(2)=1} \text{fib}(4) = 2 + 1 = 3 \\ \text{fib}(3) &= \text{fib}(2) + \text{fib}(1) \xrightarrow[\text{fib}(2)=1]{\text{fib}(1)=1} \text{fib}(3) = 1 + 1 = 2 \\ \text{fib}(2) &= \text{fib}(1) + \text{fib}(0) \xrightarrow[\text{fib}(0)=0]{\text{fib}(1)=1} \text{fib}(2) = 1 + 0 = 1 \end{aligned}$$

پاسخ ۶.۳

برج هانوی

سه میله A, B, C وجود دارد، که n مهره بر روی میله A قرار دارد، هدف انتقال n مهره به میله C است، که برای اینکار از میله B کمک گرفته میشود.



اندازه مهره ها در میله A از پایین به بالا کاهش میابد.
در انتقال میله ها هرگز مهره بزرگتر بر روی مهره کوچکتر نباید قرار بگیرد و در هر بار فقط انتقال یک مهره وجود دارد که از بالای میله انتخاب می شود.
هدف اصلی برج هانوی در واقع شکستن موارد حرکت به حرکت های کوچکتر است، الگوریتم انتقال n مهره از میله A به میله C به کمک میله B :

```
1 tower(n, A, B, C){  
2     if(n==1)  
3         A to C;  
4     else{  
5         tower(n-1,A,C,B);  
6         A to C;  
7         tower(n-1,B,A,C)}
```

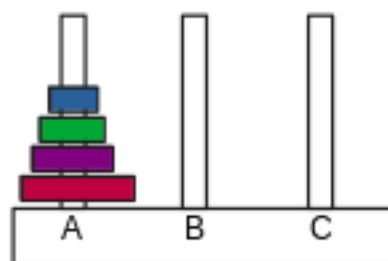
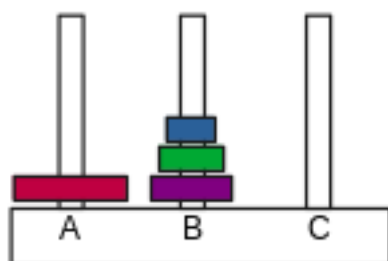
مثال ۷.۳ در برج هانوی، در صورت وجود بیش از یک مهره مسئله زیر را حل کنید.

۱. انتقال $n-1$ مهره از میله A به میله B

۲. انتقال مهره n ام از میله A به میله C

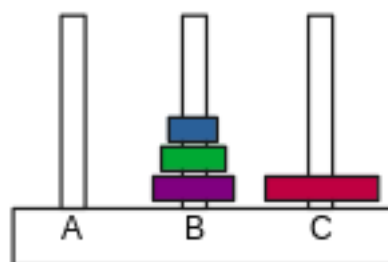
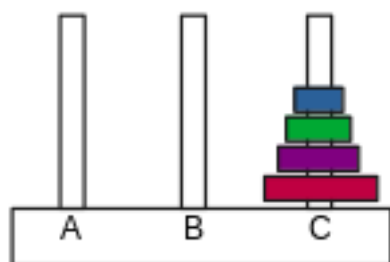
۳. انتقال مهره $n-1$ از میله B به میله C

پاسخ ۷.۳



(ب) بخش ۱ سوال

(آ) حالت اولیه



(د) بخش ۳ سوال

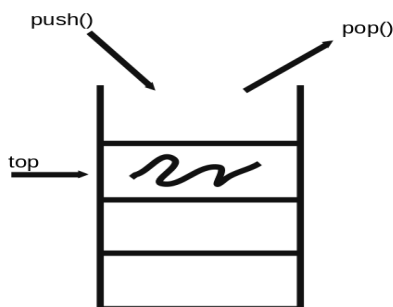
(ج) بخش ۲ سوال

مثال ۸.۳ انتقال ۳ مهره از میله A به میله C با کمک میله B برای تابع $T(3, A, B, C)$.

$$\text{پاسخ ۸.۳} \quad \left\{ \begin{array}{l} T(2, A, C, B) = \left\{ \begin{array}{l} T(1, A, B, C) = A \rightarrow C \\ A \rightarrow B \\ T(1, C, A, B) = C \rightarrow B \end{array} \right. \\ A \rightarrow C \\ T(2, B, A, C) \left\{ \begin{array}{l} T(1, B, C, A) = B \rightarrow A \\ B \rightarrow C \\ T(1, A, B, C) = A \rightarrow C \end{array} \right. \end{array} \right.$$

۲.۳ کاربرد پشته در زیر برنامه های بازگشتی

پشته ساختمان داده ای است که حذف و درج از بالای آن انجام میشود. از این خاصیت که به آن stack میگویند، توابع بازگشتی استفاده میکنند.



مثال ۹.۳ خروجی برنامه زیر بازای فراخوانی $f(1)$ چیست؟

```

1 f(n){
2     if(n==3)
3         exit();
4     else{
5         n = n + 1;
6         f(n);
7         cout<<n;}
8 }
```

پاسخ ۹.۳ تابع $f(n)$ خودش ، خودش را فراخوانی کرد، فرق اینجاست که بعد از فراخوانی دستوری وجود دارد که مقداری را چاپ میکند ، می‌خواهیم مشاهده کنیم بعد از $f(n)$ یعنی بازگشتی شدن تابع ، تکلیف دستور بعد چه خواهد بود. با توجه به اینکه ورودی تابع عدد ۱ میباشد داریم:

$$f(1) \rightarrow f(2) \rightarrow f(3)$$

در واقع $f(1)$ منجر به فراخوانی $f(2)$ شده و $f(2)$ نیز منجر به فراخوانی $f(3)$ میشود و به دلیل وجود $n = n + 1$ در `else` زمانیکه $f(3)$ میشود وارد دستور `if` شده و دیگر تابع ادامه نیافته و از برنامه خارج میشویم. برای دستور `cout<<n` عدد ۲ به عنوان اولین مقدار وارد پشته میشود ، در فراخوانی بعد یعنی $f(2)$ ، $n=3$ به عنوان مقدار دوم در پشته قرار میگیرد و چون $f(3)$ برای ما `exit` را به همراه دارد لذا محتوبات پشته بیرون خواهند آمد و خروجی ما برابر با مقدار ۳۲ میباشد.

۳
۲

مثال ۱۰.۳ خروجی برنامه زیر بازای فراخوانی $f(4)$ چیست؟

```

1 f(n){
2     if(n>2)
3     {
4         f(n-1);
5         cout<<"a";
6         cout<<"b";}
7 }
```

پاسخ ۱۰.۳ با توجه به اینکه ورودی تابع عدد ۴ میباشد داریم:

$$f(4) \rightarrow f(3) \rightarrow f(2)$$

با توجه به توقف تابع در ۲ قرار گیری دستورات درون پشته به صورت زیر خواهد بود.

a
b
a
b

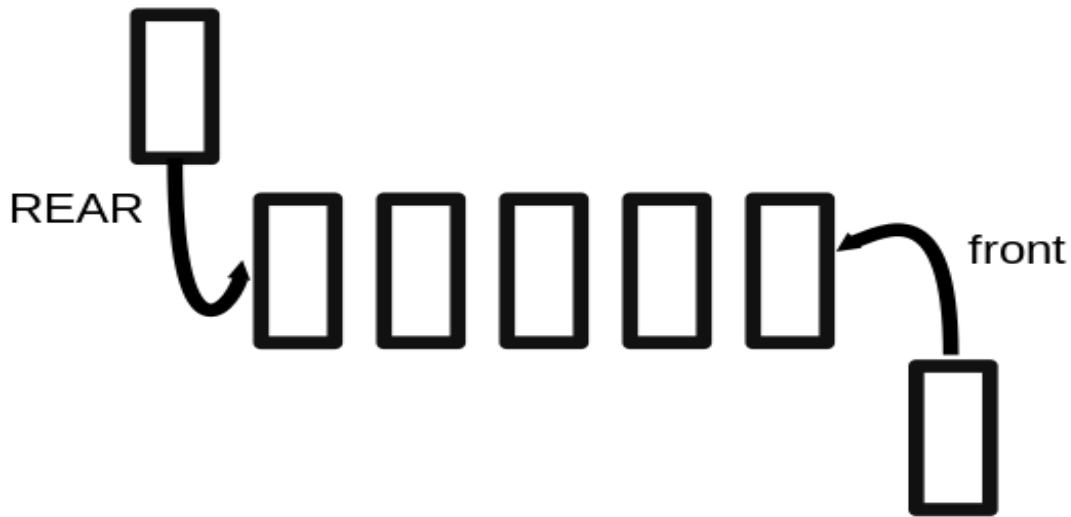
در پشته ابتدا و `cout<b` و سپس دستور `cout<a` و به همین ترتیب برای فراخوانی های بعد(بعد دلیل ترتیب دستور ها و چاپ آنها در خروجی).

لذا خروجی این تابع به صورت زیر است:

abab

۴ صف queue

ساختمان داده ای است که عمل حذف از ابتدای آن و درج به انتهای آن انجام میشود. برای پیاده سازی صف نیاز به یک آرایه یک بعدی داریم.



ابتدای صف را با متغیری به نام **front** و انتهای صف را با متغیری به نام **rear** نشان میدهیم. صف را (First In First OUT) FIFO می نامند زیرا اولین عنصر وارد شده به صف ، اولین عنصری است که از صف خارج می شود.

برای نمایش صف، از یک آرایه یک بعدی $queue[0 \dots n-1]$ و دو متغیر **front** و **rear** استفاده میشود، محل قرار گیری این دو متغیر در آرایه به صورت زیر است:
متغیر **front** یک خانه قبل تر از مقدار محل عنصر اول.
متغیر **rear** در محل آخرین عنصر صف.

مثال ۱.۴ به صفی که دارای ۴ خانه به ترتیب مقادیر A,B,C وارد میشود و سپس دو ورودی A,B از صف خارج میشوند وضعیت صف را تحلیل کنید.

0	1	2	3	information
				front = -1, rear = -1(queue is empty)
A				front = -1, rear = 0(add A to queue)
A	B			front = -1, rear = 1(add B to queue)
A	B	C		front = -1, rear = 2(add C to queue)
	B	C		front=0, rear = 2(delete A from queue)
		C		front=1, rear=2(delete C from queue)

پاسخ ۱.۴

در هنگام درج ، front تغییر نمی‌کند، آنچه تغییر میکند rear است که یک واحد اضافه می‌شود.
در هنگام حذف ، rear تغییر نمی‌کند، بلکه آنچه تغییر میکند، front است که یک واحد اضافه می‌شود.

۱.۴ درج در صف

این الگوریتم مقدار مورد نظر را در صف درج می‌کند. در صورتیکه $rear = n-1$ باشد ، آرایه تکمیل بوده و نمیتوان مقدار جدیدی وارد آرایه کرد .

در غتر این صورت rear یک گام به جلو رفته و سپس مقدار جدید را در آن محل قرار میدهد.

```

1 addq(int *rear, int item){
2     if(rear>max-1)
3         return queue_full;
4     queue[++*rear] = item;
5 }
```

در صف پر rear برابر $n-1$ میباشد و درج ممکن نیست.
در صف خالی مقدار front با rear برابر است.

۲.۴ حذف از صف

```

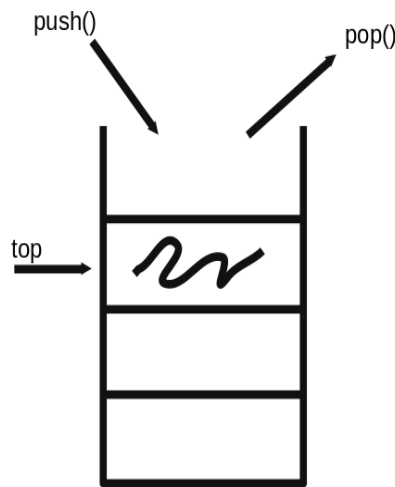
1 addq(int *front, int rear){
2     if(*front == rear)
3         return queue_empty;
4     queue[++*front];
5 }
```

در صورتیکه $front = rear$ باشد، صف خالی است.

نکته در صف عمل شیفت نداریم، اگر عنصری حذف شود همه عناصر موجود در صف یک گام به جلو بروند، front و rear هستند که تغییر میکنند زیرا عمل شیفت یک عمل زمانبر است.

۵ پشته Stack

پشته ساختمان داده ای است که عمل حذف و اضافه از بالای آن انجام می شود. پشته را (Last In First Out) LIFO می نامند زیرا آخرین عنصر وارد شده در آن، اولین عنصری که از آن برداشته میشوند. ساختار یک پشته به صورت زیر است.

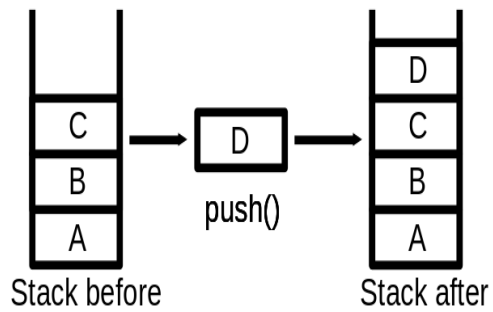


متغیر **top** مشخص کننده عنصر بالایی پشته است. در یک پشته یک آرایه داریم به همراه یک متغیر **top**، که یک سمت این آرایه بسته است و ورود و خروج اطلاعات فقط از یک سمت صورت میگیرد. در زمانی که پشته خالی است $top = -1$ می باشد. عمل درج را در پشته **push()** و عمل حذف یک عنصر از درون پشته را **pop()** می نامند.

۱.۵ درج در پشته

```
1 push(int *top, int item){
2     if(*top >= max-1)
3         return stack_full;
4     stack[++*top]=item;
5 }
```

متغیر **max** ظرفیت پشته را نمایش میدهد، لذا در صورتیکه که $max = n-1$ شود، آرایه ما پر شده است و فضای خالی ندارد. شکل تصویری درج مقدار **D** به پشته به صورت زیر است.



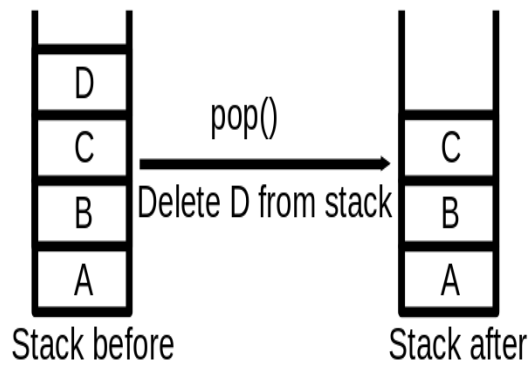
۲.۵ حذف از پشته

```

1 pop(int *top){
2     if(*top == -1)
3         return stack_empty();
4     return stack[*top--]
5 }

```

در صورتیکه $top = -1$ باشد، پشته خالی است و تابع، `stack-empty` را بر میگرداند.
در غیر این صورت مقدار خانه `top` را برگشت بده و سپس از متغیر `top` یک واحد کم کن.



۳.۵ نگارش مختلف عبارت های ریاضی

یکی از کاربرد های پشته تبدیل نگارش های مختلف ریاضی به یکدیگر میباشد.
یک عبارت ریاضی از عملگر ها و عملوند ساخته شده است که میتوان این عبارات به سه شکل نمایش داد.

۱. prefix پیشوندی : در این حالت عملگر قبل از عملوند ها قرار دارد. $*AB$

۲. infix میانوندی : در این حالت عملگر بین عملوندها قرار دارد. $A * B$

۳. postfix پسوندی: در این حالت عملگر بعد از عملوندها قرار میگیرد. $AB *$

میتوانیم هر یک از فرمت های فوق را به هم تبدیل کنیم، مثل تبدیل فرم infix به prefix یا
در بعضی از این تبدیل ها از پشته استفاده میشود.

الویت بین عملگر های دودویی

۱. توان

۲. ضرب و تقسیم

۳. جمع و تفریق

تبدیل infix به postfix

مثال زیر نحوه این تبدیل را شرح داده است.

مثال ۱.۵ عبارت زیر را از infix به postfix تبدیل کنید.

$$((A + B) * D)^{(E - F)}$$

پاسخ ۱.۵ مرحله اول : از داخلی ترین پرانتز شروع کرده و عبارت $A+B$ را به فرم postfix مینویسم.

$$(AB+ * D)^{(E - F)}$$

مرحله دوم : حال که $AB+$ ایجاد شد به آن دست نمی زنیم و به سراغ دومین عبارت یعنی $*D$ رفته و آنرا به فرم postfix مینویسم .

$$AB+D* ^{(E - F)}$$

مرحله سوم : به سراغ دومین پرانتز یعنی $E - F$ رفته و آنرا به فرم postfix در می آوریم .

$$AB+D* ^{EF-}$$

مرحله چهارم: در مرحله آخر برای عملگر توان و عملوند های بین آن نیز تبدیل به postfix را انجام میدهیم و فرآیند تبدیل به اتمام میرسد.

$$AB+D*EF-^$$

نکته در نگارش prefix و postfix یک عبارت ، نیازی به پرانتز گذاری نیست و ترتیب عملگر ها از جایگاه آنها مشخص میشود.

تبدیل infix به prefix

مثال ۲.۵ عبارت زیر را از infix به prefix تبدیل کنید.

$$((A + B) * D)^{(E - F)}$$

پاسخ ۲.۵ مرحله اول: از داخلی ترین پرانتز شروع کرده و عبارت $A+B$ را به فرم prefix مینویسم.

$$(+AB * D)^{(E - F)}$$

مرحله دوم: سپس برای $*D$ ، $*$ به قبل از $+$ برده و D در جای باقی می ماند.

$$*+ABD ^{(E - F)}$$

مرحله سوم: برای پرانتز دوم نیز همین تبدیل را انجام داده و عملگر را به پشت عملوند ها میبریم.

$$AB+D* ^{-}EF$$

مرحله چهارم: در مرحله آخر با قرار دادن علامت $^$ به قبل از عملگر $*$ عملیات تبدیل به پایان میرسد.

$$^*+ABD-EF$$

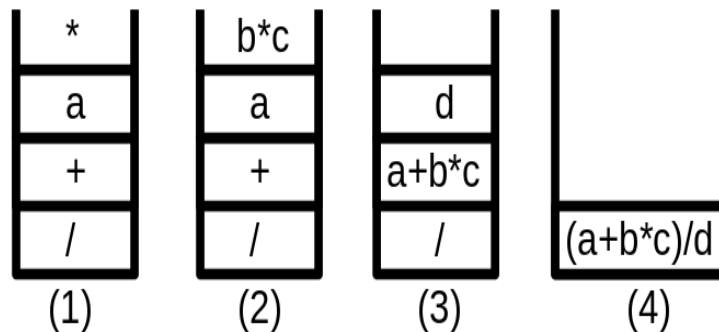
تبدیل prefix به infix با استفاده از پشته

مثال ۳.۵ عبارت زیر را از prefix به infix تبدیل کنید.

$$/+a*bcd$$

پاسخ ۳.۵ مراحل تبدیل با کمک پشته به صورت زیر است.

با کمک خاصیت های push و pop یک پشته میتوانیم مقدار مورد نظر که یک عبارت infix باشد را برای یک عبارت prefix بدست آوریم.

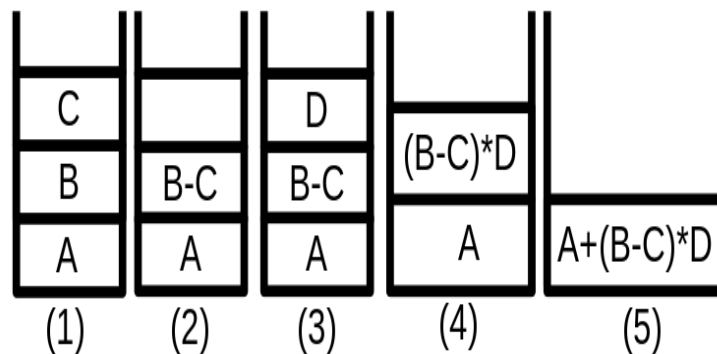


مرحله اول: از سمت چپ به راست شروع کرده و اول / سپس * و تا زمانی که به دومین عملوند (b) برسیم ، مقادیر را به ترتیب درون پشته قرار میدهیم.(شکل ۱)

نکته به دلیل تقدم عملگر ها برای آخرين عبارت از پرائنتر استفاده شد زير ابتدا بايد $b * c$ شده و بعد با a جمع و در نهايت تقسيم بر d شود.

تبدیل postfix به infix

ABD -D*+

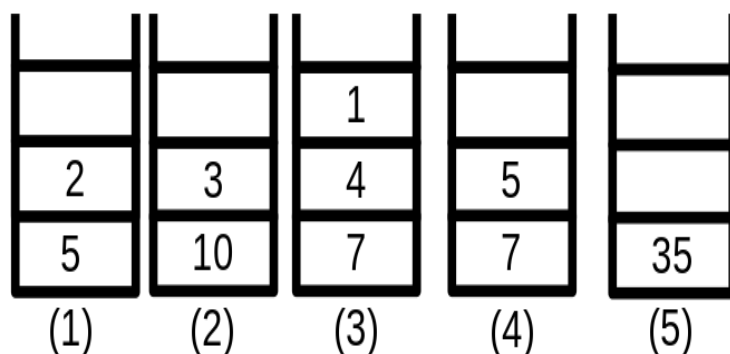


مرحله اول : عملوند ها را به ترتیب از چپ به راست درون پشته قرار میدهیم تا به اولین عملگر (-) برسیم (پشته ۱)
 مرحله دوم : دو عنصر بالایی پشته را برداشته و عملگر را روی آنها اعمال میکنیم. (پشته ۲)
 مرحله سوم : در این مرحله دوباره عملوند ها را در پشته قرار می دهیم تا به اولین عملگر (*) برسیم. (پشته ۳)
 مرحله ۴ : در این مرحله باید دو عبارت بالایی پشته را برداشته و عملگر را روی آنها اعمال دهیم. (پشته ۴)
 در این مرحله دقت شود که عبارت B-C دارای تقدم در محاسبات میباشد لذا درون پرانتز قرار میگیرد.
 مرحله ۵ : در مرحله آخر دوباره درون عبارت شروع به حرکت میکنیم و به عملگر + میرسیم لذا عبارت پایین تر درون پشته را با عبارت بالایی پشته جمع میکنیم و حاصل پیدا خواهد شد. (پشته ۵)

مثال ۵.۵ حاصل عبارت زیر را بدست آورید. (عبارت postfix است آنرا به infix نوشته و حاصل را بدست آورید.

5, 2, *, 3, 4, 1, +, *

پاسخ ۵.۵ نحوه حل به صورت زیر است.



در گام نخست دو عملوند اول را درون پشته قرار میدهیم تا به اولین عملگر برسیم. (تصویر ۱)
 به علامت * رسیدیم لذا $5*2$ کرده و حاصل را در خانه اول پشته ۲ قرار میدهیم ، سپس عملوند ها را قرار داده تا به اولین عملگر برسیم . (تصویر ۲)
 در مرحله بعد عملگر - را بر روی دو عبارت درون پشته اثر میدهیم و حاصل یعنی عدد ۷ را درون اولین خانه پشته ۳ قرار میدهیم و دوباره حرکت کرده تا به اولین عملگر برسیم. (پشته ۳)
 به عملگر + رسیدیم لذا این عملگر را بر روی دو خانه بالایی پشته اثر داده و حاصل را در خانه دوم پشته ۴ میگذاریم و برای خانه اول پشته ۴ مقدار خانه اول پشته ۳ قرار میگیرد عدد ۷. (تصویر ۴)
 در مرحله آخر به عملگر * رسیدیم و این عملگر را بر روی دو عملوند پشته ۴ اثر داده و حاصل این عبارت پیدا خواهد شد عدد ۳۵ (تصویر ۵)

نکته

به همان ترتیبی که اعداد در سوال قرار گرفته اند، از چپ به راست حرکت کرده و مقادیر را در پشته قرار میدهیم.
 اگر به عملگری رسیدیم ترتیب قرار گرفتن عملوند ها به صورتی است که ابتدا عملوند پایین تر و بعد قرار گرفتن عملگر و در نهایت قرار گرفتن عملوند بالاتر .

۶ لیست پیوندی

ساختمان داده هایی که تاکنون به مطالعه پرداختیم همگی از جنس ایستا بوده اند. لیست پیوندی یک ساختمان داده پویاست، که اشیا با ترتیب خاصی در آن قرار گرفته اند. زمانی که میگوییم پویا، یعنی میتوان عناصر را به تعداد دلخواه درج و کرد و محدودیتی از این جهت در معرفی تعداد عناصر در ابتدا نداریم.

در هنگام تعریف یک آرایه باید تعداد عناصر را مشخص کنیم و در مراحل انجام کار از این تعداد عناصر استفاده شده استفاده کنیم، و برای اینکه بخواهیم بیشتر از حد تعریف شده از یک آرایه استفاده کنیم این امکان وجود نداشته و به همین دلیل میگوییم آرایه نوع داده ای ایستا میباشد. اما در لیست پیوندی هرگاه نیاز باشد میتوانیم عنصری را اضافه کرده و حالتی پویا دارد. برخلاف آرایه که در آن ترتیب خطی توسط اندیس های آرایه تعریف میشود، ترتیب در لیست پیوندی بوسیله یک اشاره گر در هر شی تعیین میگردد، بنابر این دسترسی به عناصر آرایه برای ما راحت تر است.

لیست پیوندی دارای دو نوع می باشد.

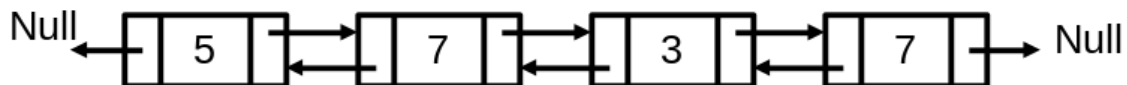
۱. لیست پیوندی یک طرفه:

در لیست پیوندی یک طرفه حرکت ما از یک سمت است و تا زمانی ادامه می یابد که اشاره گر دیگر به هیچ گره ای اشاره نمیکند. (Null)



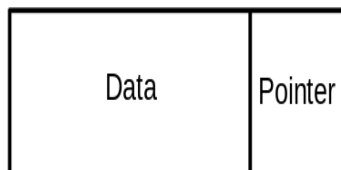
۲. لیست پیوندی دو طرفه:

حرکت در لیست پیوندی دو طرفه هم به سمت جلو و هم عقب میباشد (در دو جهت حرکت داریم) و همانند لیست پیوندی یک طرفه این حرکت تا زمانی است که اشاره گر دیگر به هیچ گره ای اشاره نکند.

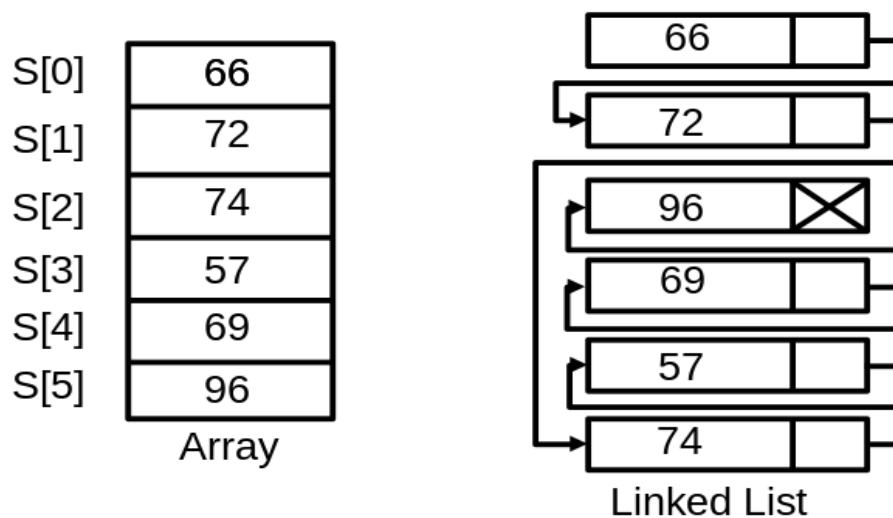


بررسی ساختار آرایه پویا

به هر عنصر در آرایه پویا یک گره میگویند که دارای دو قسمت زیر میباشد.
 قسمت اول که در آن دیتا یا داده ما قرار دارد.
 قسمت دوم که در آن اشاره گر وجود دارد که نشان میدهد بعد از این گره چه گره ای را داریم (اشاره به گره بعدی دارد) تصویر یک گره در آرایه پویا به صورت زیر میباشد.



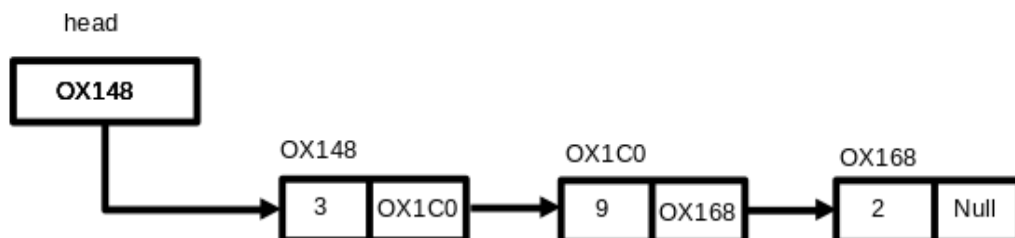
مقایسه آرایه با لیست پیوندی



۱. در لیست پیوندی عناصر پشت سر هم نیستند و هر مقدار در هر جایی که سیستم عامل تشخیص دهد قرار میگیرد، این اشاره گر است که معلوم میکند بعد از هر عنصر باید کجا برویم.
۲. دسترسی به عناصر آرایه به نسبت لیست پیوندی سریع تر است، زیرا اندیس هر عنصر در آرایه مشخص است.
۳. درج و حذف در لیست پیوندی نسبت به آرایه ساده تر است، زیرا نیاز به جابه جایی فیزیکی (شیفت) عناصر در حافظه نمی باشد.
۴. امکان جستجو دودویی در لیست پیوندی وجود ندارد و در لیست پیوندی فقط جستجو خطی را داریم.

۱.۶ لیست پیوندی یک طرفه

لیست پیوندی یک طرفه، لیست است که در آن، هر عنصر فقط آدرس عنصر بعدی را نگه داری میکند. هر یک از گره های این لیست پیوندی از دو قسمت داده و آدرس تشکیل شده اند.



به طور مثال لیست پیوندی فوق دارای ۳ گره است، گره هایی با دیتای ۳ و ۹ و ۲. همچنین همانگونه که پیداست هر گره دارای آدرس گره بعدی میباشد.

اولین گره first یا head نام دارد که آدرس در یک متغیر به نام head ذخیره میشود.

در زبان سی C گره یک structure است که دارای دو بخش (دیتا و آدرس) میباشد و نحوه تعریف یک گره یا node در زبان سی به صورت زیر است:

```

1 struct node{
2     int data;
3     struct node *next;}

```

لازم به ذکر است که *next آدرس گره است، که این آدرس باید یک اشاره گر باشد.
چاپ محتویات لیست پیوندی یک طرفه

```

1 print_list(head){
2     p = head;
3     for(;p!=Null;p=p->next)
4         cout<<p->data
5 }

```

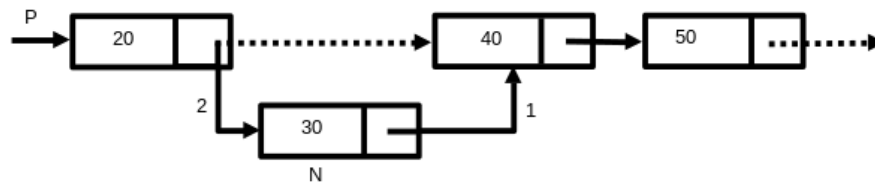
تابع فوق داده های لیست head را چاپ میکند.

تابع head را دریافت میکند که آدرس گره اول است .

و در این تابع یک حلقه داریم که تا زمانی که p!=Null نشده این حلقه ادامه یافته و دیتای p (p->data) را در خروجی نمایش میدهد.

دستور p->next اشاره گر را یک گره به جلو می بردو لازم به ذکر است که این دستور برای اشاره گر ها میباشد زیرا در لیست پیوندی ترتیب نداریم که بخواهیم از شمارنده استفاده کنیم.

درج یک گره در لیست پیوندی یک طرفه



تابع زیر یک گره با داده *item* را بعد از یک گره با آدرس مشخص *p* درج میکند. *p* گره ای است که برای ما آدرسش مشخص است و می‌خواهیم گره ای را بعد از آن ایجاد کنیم.

```

1 insert(head,p,item){
2     n = malloc(sizeof(node));
3     n->data = item;
4     n->next = p->next;(1)
5     p->next = n;(2)
6 }
```

`n=malloc(sizeof(node))`

در مرحله اول با دستور فوق، یک فضا برای گره ای که می‌خواهیم ایجاد کنیم از حافظه می‌گیریم، این فضا به اندازه یک گره است و آدرسی که بر می گرداند را *n* می نامیم.

`n->data=item`

با این دستور در قسمت دیتا گره ایجاد شده مقدار *item* را قرار می دهیم.

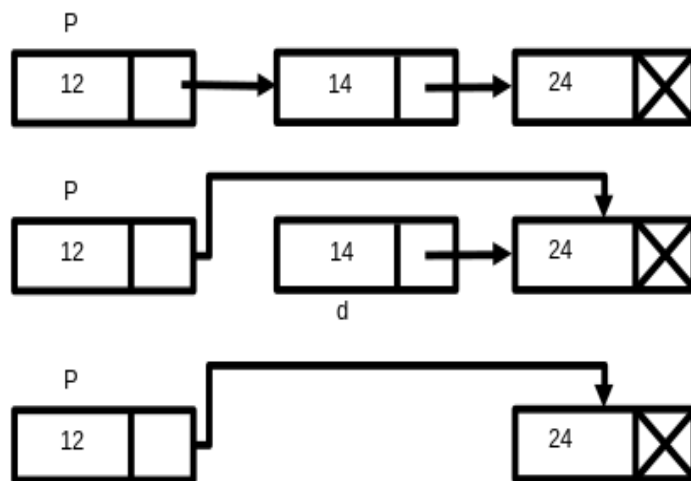
`n->next = p-> next`

حال به اشاره گر *n* میرسیم ، باید به آن مقدار دهیم، به همین دلیل در قسمت *next* آن *p->next* را قرار میدهم به این معنی که اگر اشاره گر عنصر *p* ام به *p->next* اشاره داشت، حال اشاره گر *n* نیز به *p->next* اشاره میکند.

`p->next=n`

حال برای اینکه عنصر *p* به گره *n* اشاره کند از این دستور استفاده میکنیم و *p->next* را برابر گره *n* قرار میدهم تا این ارتباط شکل گیرپ و در نهایت کار به اتمام برسد.
در چیدمان این خطوط بسیار دقت شود.

حذف یک گره در لیست پیوندی یک طرفه



تابع زیر گره با آدرس d را حذف میکند، گره d بعد از گره با آدرس مشخص p قرار دارد.

```

1 delete(first,p,d){
2     if(p)
3         p->next = d->next;
4     else
5         *first = (*first)->next;
6     free(d);
7 }
```

برای حذف یک گره مثل d کافیست اشاره گر p به جای اشاره به d به عنصر بعدی d یعنی d->next اشاره کند.

if(p)

در صورت وجود گره p

p->next = d->next

اشاره گر p به گره بعد از d اشاره کند.

در غیر اینصورت گره ای میخواهد حذف شود، عنصر اول لیست پیوندی است و در واقع میخواهیم گره اول را حذف کنیم.

*first = (*first)->next

در این حالت اشاره گر شروع لیست پیوندی را یک آدرس جلو میبریم و با این کار گره اول را حذف میکنیم.

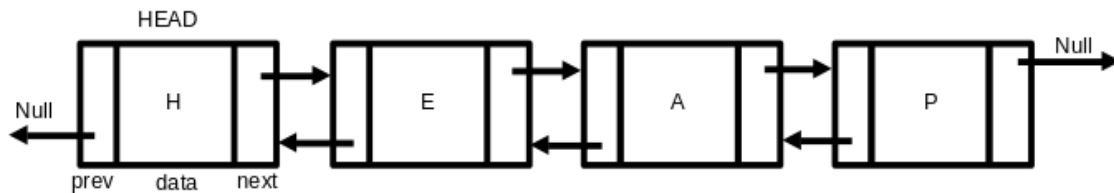
free(d)

در نهایت باید گره d را رها یا آزاد کنیم تا از فضا حذف شده و به لیست گره های آزاد بپیوندد.

۲.۶ لیست پیوندی دو طرفه

در لیست های پیوندی دو طرفه (دو سویه) هر عنصر علاوه بر مولفه های لیست یک طرفه، مولفه `prev` نیز دارد، که به عنصر قبلی خود اشاره میکند.

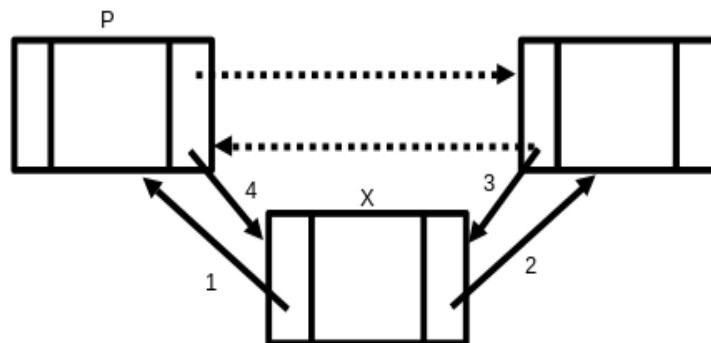
در لیست پیوندی دو طرفه با استفاده از اشاره گر ها امکان حرکت به دو طرف را داریم، بنابراین با داشتن آدرس یک گره به کلیه گره های لیست میتوان دسترسی داشت.



تعریف یک گره در لیست پیوندی دو طرفه

```
1 struct node{
2     int data;
3     struct node *next;
4     struct node *prev;
5 }
```

اضافه کردن گره به در لیست پیوندی دو طرفه
اضافه کردن گره به سمت راست گره ای با آدرس مشخص `p`.



```

1 insert(p,x){
2   x->prev = p;(1)
3   x->next = p->next;(2)
4   p->next->prev = x;(3)
5   p->next = x;(4)
6 }

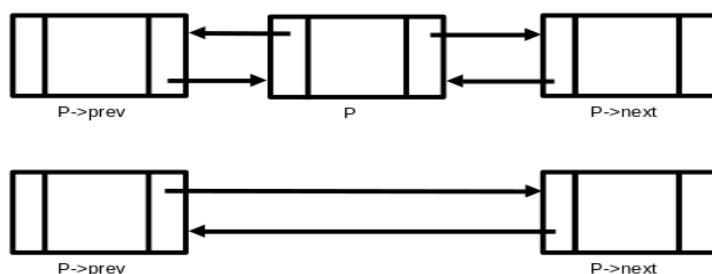
```

برای اضافه کردن یک گره به لیست پیوندی دو طرفه، به ۴ تغییر اشاره گر نیاز داریم. برای اینکار باید اسم یک گره را بدانیم (اگر اسم گره p باشد گره بعد از p را $p \rightarrow next$ خواهد بود). مراحل این انتقال و در کد و تصویر شماره گذاری شده اند و به شرح زیر میباشد.

- (1) بخش $prev$ گره x به گره p اشاره میکند.
- (2) گره x از سمت $next$ باید به گره $p \rightarrow next$ اشاره کند.
- (3) گره $p \rightarrow next \rightarrow prev$ از سمت چپ به x اشاره کند.
- (4) $p \rightarrow next$ از سمت راست به گره x ، اشاره کند (به عبارتی آخرین کاری که انجام میدهیم، ارتباط p را از سمت راست قطع میکنیم).

حذف یک گره از لیست پیوندی دو طرفه

تابع زیر گره با آدرس مشخص p را از لیست پیوندی دو طرفه حذف میکند، برای اینکار کافیست اشاره گر $next$ گره قبل از p به گره بعد از p اشاره کند و اشاره گر $prev$ بعد از p به گره قبل از p اشاره کند. برای حذف یک گره از لیست پیوندی دو طرفه، به ۲ تغییر اشاره گر نیاز داریم.



```

1 delete(p){
2   p->prev->next = p->next;
3   p->next->prev = p->prev;
4   free(d);
5 }

```

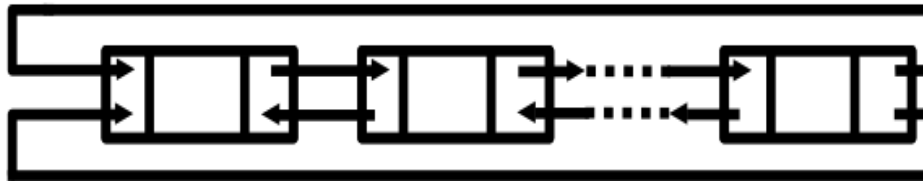
۳.۶ لیست پیوندی حلقوی

لیست پیوندی (حلقوی) نوعی لیست یک طرفه است که در آن اشاره گر آخرین گره، به جای مقدار با Null به ابتدای لیست اشاره میکند. در لیست چرخشی با داشتن آدرس هر گره میتوان به کلیه گره ها دسترسی داشت. مزیت لیست یک طرفه چرخشی نسبت به لیست یک طرفه غیر چرخشی در این است که به گره قبلی نیز دسترسی داشت.

۱. لیست پیوندی یک طرفه چرخشی



۲. لیست پیوندی دو طرفه چرخشی



محاسبه طول یک لیست حلقوی

این تابع آدرسی در لیست یک طرفه چرخشی دریافت میکند و به ما طول لیست (تعداد گره ها) را خواهد داد.

```
1 length(p){
2     c = 0;
3     if(p){
4         t = p;
5         do{
6             c++;
7             t=t->next;
8         }while(t!=p)
9     }
10    return C;
11 }
```

۴.۶ پیاده سازی پشته با لیست پیوندی

پشته لیستی است که حذف و درج از ابتدای آن انجام میشود. برای نمایش پشته در لیست پیوندی گره ها را از آخر به اول قرار میدهیم. شرط اصلی پیاده سازی پشته به کمک لیست پیوندی، در این است که اگر بخواهیم مقداری را به لیست پیوندی اضافه کنیم باید به ابتدای لیست اضافه شود و برای حذف یک مقدار نیز باید عمل حذف از اول لیست پیوندی انجام شود.

الگوریتم درج به پشته ایجاد شده با لیست پیوندی

```
1 add(top, item){
2     n = malloc(sizeof(stack));
3     n->data = item;
4     n->next = top;
5     top = n;
6 }
```

الگوریتم حذف از پشته ایجاد شده با لیست پیوندی

```
1 delete(top){
2     d = top;
3     if (d==Null) exit();
4     else{
5         item = d->data;
6         top = d->next;
7         free(d);
8         return item;
9     }
10 }
```

۵.۶ پیاده سازی صف به کمک لیست پیوندی

صف لیستی است که به انتهای آن و حذف از ابتدای آن انجام می شود. اولین گره لیست، عنصر ابتدایی صف خواهد بود.

الگوریتم درج به صف ایجاد شده با لیست پیوندی

```
1 addq(front, rear, item){
2     n = malloc(sizeof(queue));
3     n->data = item;
4     n->next = Null;
5     if(front)
6         rear->next;
7     else
8         front = n;
9     rear = n
10 }
```

الگوریتم حذف از صف ایجاد شده با لیست پیوندی

```
1 deleteq(front){
2     if(front==Null)
3         exit();
4     d = front;
5     item = d->data;
6     front = d->next;
7     free(d);
8     return item;
9 }
```


۷ درخت

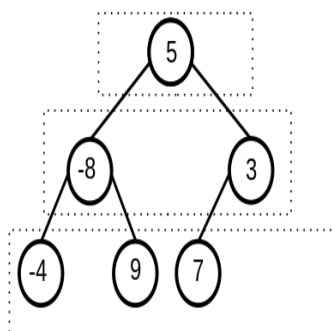
دسته بندی ساختمان داده ها به صورت زیر است:

۱. خطی: آرایه، صف، پشته، لیست پیوندی

۲. غیر خطی: درخت، گراف

۱.۷ تعاریف اولیه

درخت مجموعه محدودی از یک یا چند گره میباشد، که دارای گره خاصی به نام ریشه است و بقیه گره ها نیز به مجموعه مجزا تقسیم میشوند که هر یک از مجموعه ها خود نیز یک درخت میباشد.



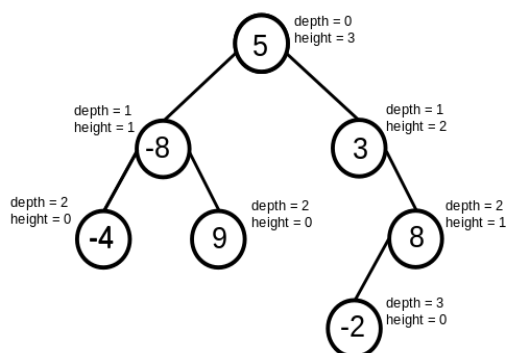
□ ریشه: گره‌ای که دارای پدر نیست.

□ برگ: گره بدون فرزند (گره با درجه ۰)

□ گره داخلی: گره غیر برگ و ریشه

□ برادر: گره‌هایی با یک پدر

ارتفاع درخت و گره



□ ارتفاع گره: طول بزرگترین مسیر از آن گره به برگ

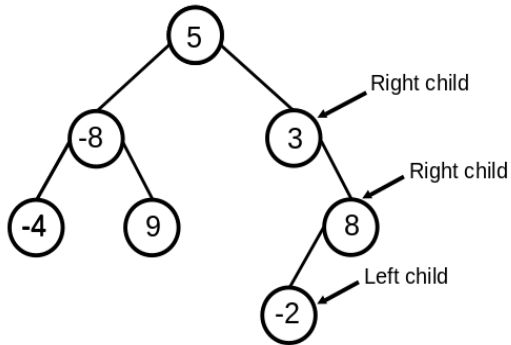
□ ارتفاع درخت: ارتفاع ریشه

□ عمق (سطح) گره: طول مسیری از ریشه به آن گره

نکته

درخت حلقه ندارد و در واقع نوعی گراف است که حلقه ندارد.

درخت دودویی



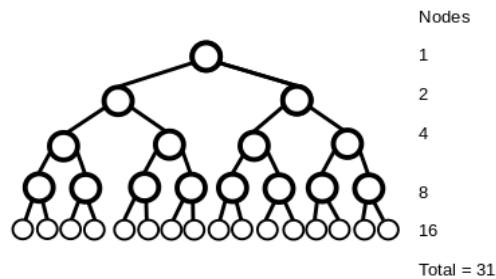
درخت دودویی، درختی مرتب ای که هر عنصر آن حداکثر دارای دو فرزند (فرزند چپ و راست) می باشد.
درخت مرتب : درختی که ترتیب فرزندان هر گره مشخص است.

درخت دودویی پر

درختی که هر گره به غیر از برگ ها دارای دو فرزند است.

مثال ۱.۷ رسم یک درخت دودویی پر با ۳۱ گره و ارتفاع ۴

پاسخ ۱.۷ این درخت به صورت زیر خواهد بود.



لذا با توجه به درخت فوق و تعداد گره ها به رابطه های زیر خواهیم رسید :
h ارتفاع درخت خواهد بود

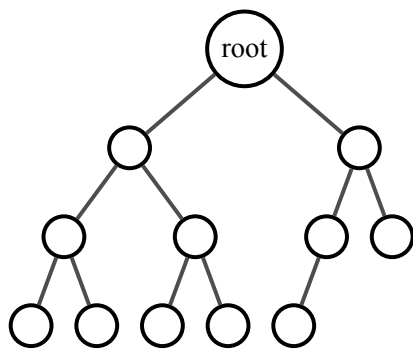
□ تعداد برگ ها : 2^h

□ تعداد گره : $n = 2^{h+1} - 1$

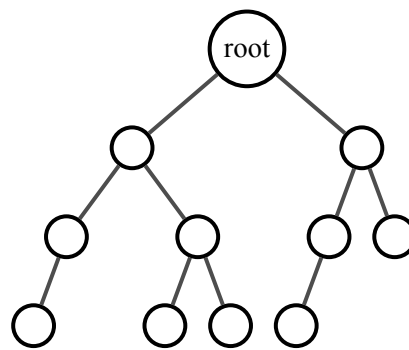
□ تعداد گره داخلی (بدون ریشه و برگ) : $\text{floor}(n/2)$

درخت دودویی کامل

هر سطح به جز احتمالا سطح آخر به طور کامل پر شده است و همه گره ها تا جایی که ممکن است در چپ درخت قرار میگیرند. درختی که پر است، کامل نیز می باشد.



درخت کامل است.



درخت کامل نیست زیرا فرزند راست ندارد

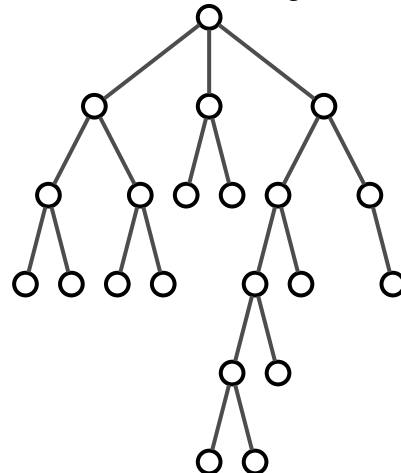
تعداد برگ ها

تعداد برگ های یک درخت k تایی از رابطه زیر محاسبه می شود:

$$n_0 = (k - 1)n_k + (k - 2)n_{k-1} + \dots + n_2 + 1$$

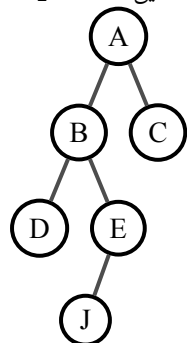
به طر مثال داریم:

درخت ۳ تایی : $n_0 = 2n_3 + n_2 + 1$



$$n_3 = 1, n_2 = 8, n_0 = 11$$

درخت ۲ تایی : $n_0 = n_2 + 1$

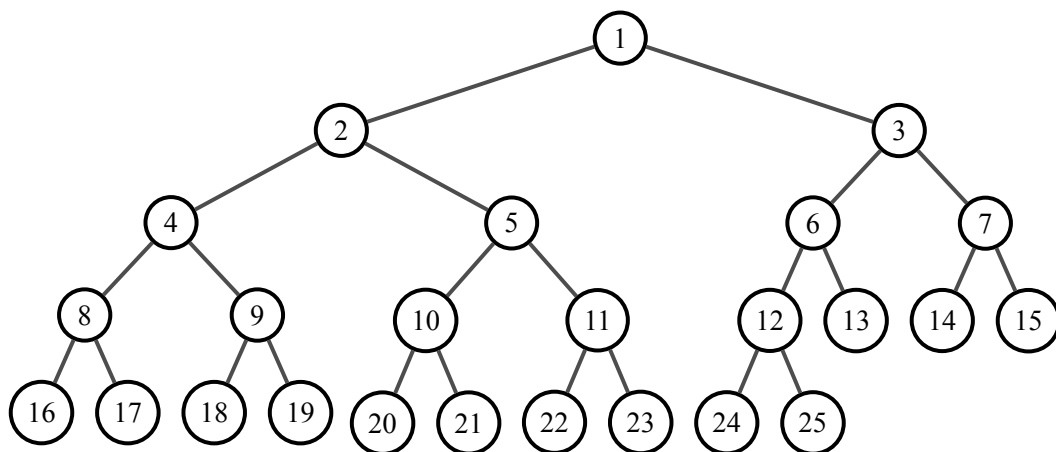


$$n_2 = 2, n_0 = 3$$

شماره گذاری

در هر درخت دودویی کامل شماره گذاری شده، برای هر گره با اندیس i قواعد زیر برقرار است:

۱. اگر $i = 1$ باشد، آنگاه ریشه است.
 ۲. اگر $i > 1$ باشد، آنگاه واحد آن در حد پایین $\frac{i}{2}$ است.
 ۳. اگر $2i \leq n$ باشد، آنگاه فرزند چپ آن در $2i$ است.
 ۴. اگر $2i+1 \leq n$ باشد، آنگاه فرزند راست آن در $2i+1$ است.
- شماره گذاری یک درخت دودویی کامل به صورت زیر میباشد.

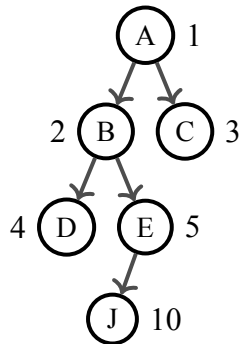


۲.۷ پیاده سازی درخت دودویی

یک درخت را می توان به کمک آرایه یا لیست پیوندی نمایش داد.

۱.۲.۷ پیاده سازی درخت دودویی با استفاده از آرایه

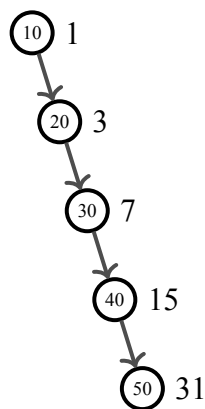
با توجه به شماره گذاری هر گره درخت دودویی و با توجه به محل قرار گیری هر گره در درخت، آنرا در خانه مربوطه در آرایه ذخیره می کنیم.



1	2	3	4	5	6	...	10	...	15
A	B	C	D	E		...	J	...	

لذا هر گره را درون آرایه در خانه ای که شماره آن مشخص است قرار می دهیم.

مثال ۲.۷ پیاده سازی درخت اریب با کمک آرایه



1	2	3	...	7	...	15	...	31
10		20	...	30	...	40	...	50

پاسخ ۲.۷

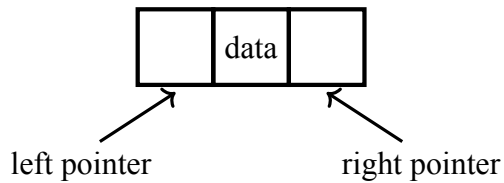
باتوجه به شماره گذاری های گره موجود، آرایه ای ایجاد می کنیم، به اندازه شماره بزرگترین گره ای که داریم و سپس در خانه با شماره گره مربوطه مقادیر را قرار می دهیم.

از مثال بالا متوجه می شویم که از ۳۱ خانه آرایه فقط ۵ خانه پر بوده است. لذا نتیجه می گیریم که بهتر است درخت اریب را با آرایه نمایش ندهیم.
آرایه بیشتر برای درخت های کامل و پر مناسب است.

۲.۲.۷ پیاده سازی درخت با لیست پیوندی

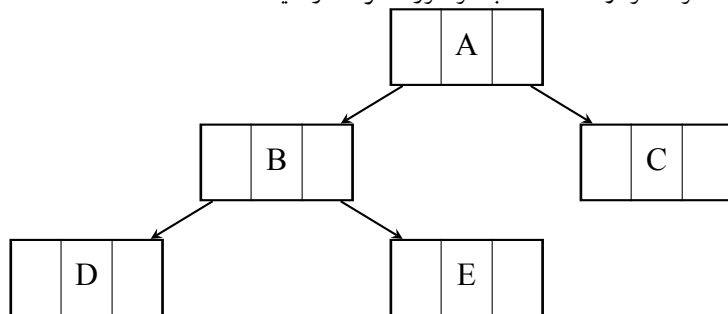
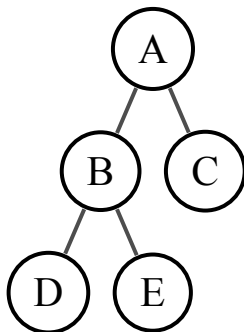
هر گره در درخت دودویی شامل ۳ قسمت داده، اشاره گر به فرزند راست و اشاره گر به فرزند چپ می باشد. (لیست پیوندی دو طرفه)
تعریف یک گره درخت دودویی در زبان C

```
1 struct node{
2     int data;
3     struct node* left;
4     struct node* right;
5 };
```



مثال ۳.۷ درخت رو به رو را به صورت لیست پیوندی پیاده سازی کنید.

پاسخ ۳.۷ در واقع هر فلش در شکل زیر یک اشاره گر از ظرف گره والد است که به گره فرزند خود اشاره میکند.



پیاده سازی درخت با لیست پیوندی باعث می شود، خانه های خالی در آرایه نداشته باشیم.
بهتر است درخت های پر و کامل را با آرایه نمایش داده و درخت اریب را بهتر است به کمک لیست پیوندی نمایش دهیم.

۳.۷ پیمایش درخت دودویی

پیمایش درخت یعنی حرکت روی یال های درخت و در این پیمایش باید با همه گره ها دقیقا یک بار ملاقات شود.

۱.۳.۷ روش های معمول پیمایش درخت دودویی

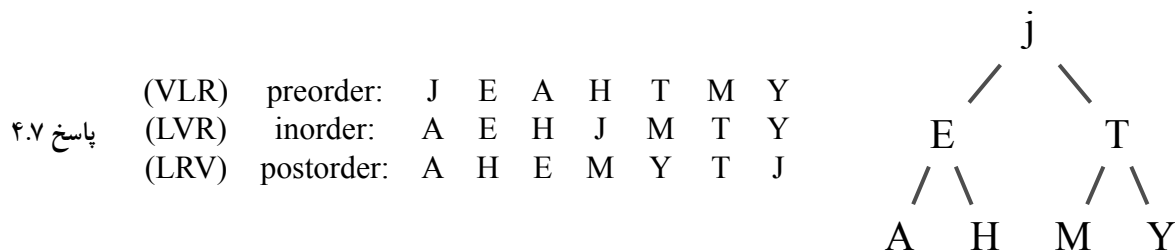
۱. پیشوندی VLR (ریشه - چپ - راست) ، این روش را preorder می نامند.

۲. میانوندی LVR (چپ - ریشه - راست) ، این روش را inorder می نامند.

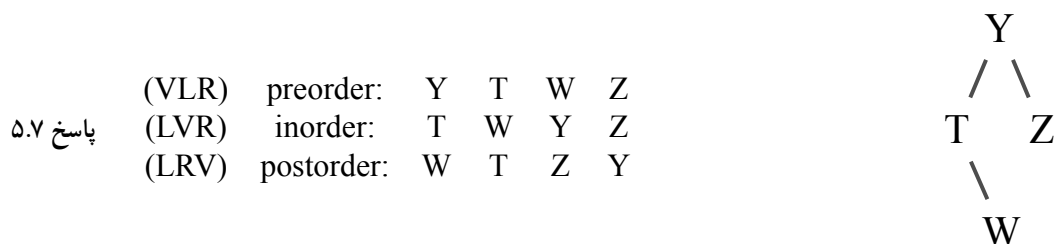
۳. پسوندی LRV (چپ - راست - ریشه) این روش را postorder می نامند.

در پیمایش پیشوندی ابتدا ریشه ملاقات می شود، سپس زیر درخت سمت چپ به روش پیشوندی ملاقات می شود و در نهایت زیر درخت سمت راست به روش پیشوندی پیمایش می شود.

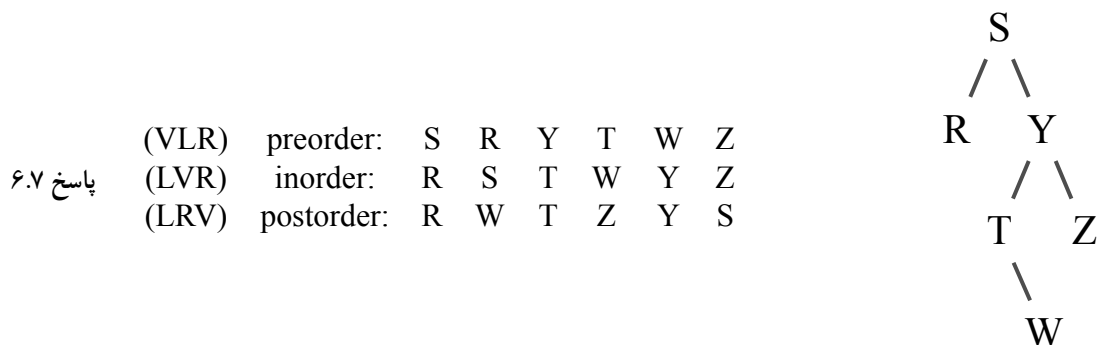
مثال ۴.۷ سه روش پیمایش زیر را برای درخت زیر انجام دهید:



مثال ۵.۷ پیمایش درخت زیر را با سه روش پیمایش درخت انجام دهید.

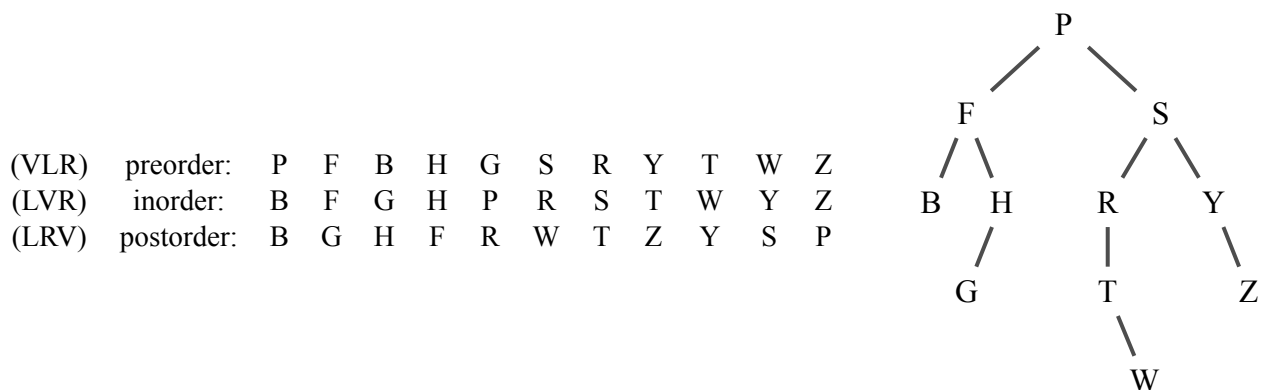


مثال ۶.۷ پیمایش درخت زیر را با سه روش پیمایش درخت انجام دهید.

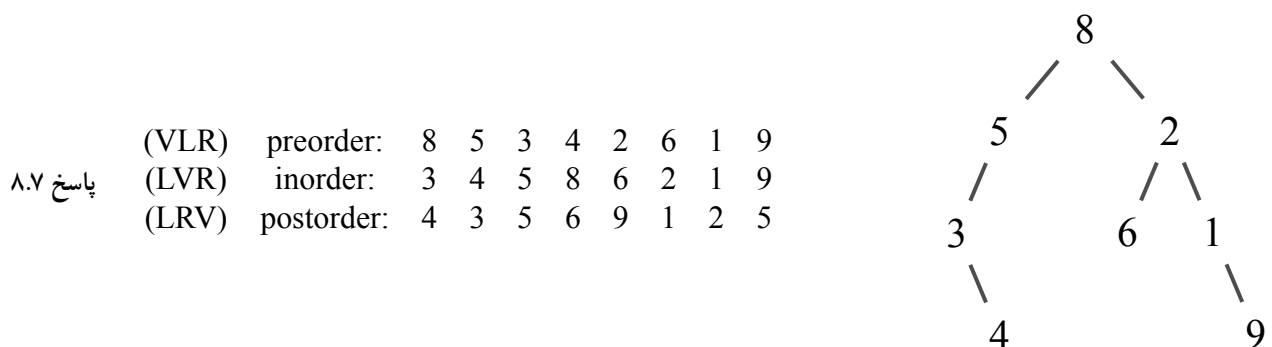


مثال ۷.۷ سه روش پیمایش درخت دودویی را برای درخت زیر انجام دهید:

پاسخ ۷.۷ با توجه به گراف زیر داریم:



مثال ۸.۷ پیمایش درخت زیر را با سه روش پیمایش درخت انجام دهید.



پیمایش یک درخت دودویی یک موضوع بازگشتی (recursive) می باشد، لذا برای انجام پیمایش یک درخت ، باید پیمایش را برای زیر درخت ها انجام دهیم.

۲.۳.۷ الگوریتم های پیمایش درخت دودویی

الگوریتم های پیمایش درخت دودویی با سه حالت preorder و inorder و postorder به صورت زیر می باشد:
توابع زیر همگی از نوع بازگشتی بوده و p آدرس ریشه درخت دودویی می باشد.

```
1 ////////////////////////////////////////////////// preorder //////////////////////////////////
2 pre(p){
3     if(p==null) return;
4     cout<< p->data;
5     pre(p->left);
6     pre(p->right);
7 }

1 ////////////////////////////////////////////////// inorder //////////////////////////////////
2 in(p){
3     if(p==null) return;
4     in(p->left);
5     cout<< p->data;
6     in(p->right);
7 }

1 ////////////////////////////////////////////////// postorder //////////////////////////////////
2 post(p){
3     if(p==null) return;
4     post(p->left);
5     post(p->right);
6     cout<<p->data
7 }
```

۴.۷ رسم درخت دودویی

۱.۴.۷ رسم درخت دودویی با داشتن پیمایش های پیشوندی و میانوندی

مثال ۹.۷ باتوجه به پیمایش های زیر ، درخت دو دویی را رسم کنید:

preorder: a b d f c e g
inorder: d f b a e g c

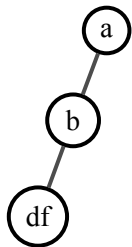
پاسخ ۹.۷ برای رسم درخت باید مراحل زیر را انجام دهیم:

گام اول) به دلیل وجود preorder مشخص می شود که حرف اول در عبارت preorder یعنی a ریشه درخت خواهد بود. حال در عبارت inorder، ریشه را پیدا می کنیم و عبارات سمت چپ و راست گراف مشخص خواهد شد.

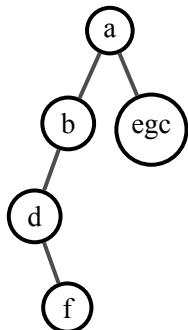


گام دوم) حال باید به عبارت های چپ ریشه بپردازیم.

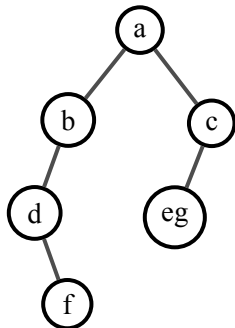
با مراجعه به عبارت preorder باید گره های عبارت سمت چپ را بیابیم، می بینیم که بعد از a ، b قرار دارد لذا نتیجه می گیریم که برای عبارت سمت چپ، b رشته می باشد.



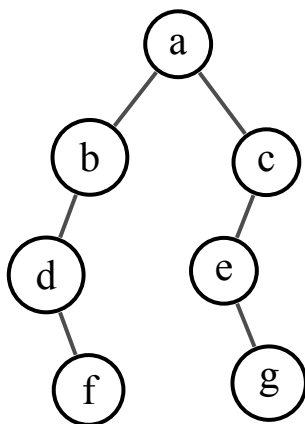
گام سوم) حال برای تعیین وضعیت دو گره d و f دوباره به عبارت preorder مراجعه میکنیم و متوجه خواهیم شد که بعد از گره b گره d وجود دارد، در نتیجه d ریشه ما نیست به f خواهد بود و با نگاه به عبارت inorder متوجه خواهیم شد که گره d قبل از گره f می باشد و گره f فرزند راست گره d خواهد بود.



گام چهارم) برای تعیین وضعیت egc، بعد از رسم کامل گره ها سمت چپ (dfb) با مراجعه به عبارت preorder و رسیدن به عبارت ceg متوجه خواهیم شد که گره c ریشه می باشد.



گام پنجم) سپس برای وضعیت eg با مراجعه به inorder، متوجه خواهیم شد که eg قبل از c می باشد لذا eg در چپ می باشد و با مراجعه به preorder متوجه خواهیم شد که e پدر گره g خواهد بود و محل قرار گیری g در سما راست است.



معمولا با داشتن دو پیمایش میتوان درخت را رسم کرد به شرط آنکه یکی از دو پیمایش ها inorder باشد.

۲.۴.۷ مشخص کردن گره های تک فرزندی

می توان به کمک دو پیمایش preorder و postorder یک درخت دودویی، گره های تک فرزندی را مشخص کرد. برای اینکار در پیمایش preorder از چپ به راست حرکت کرده و زوج پشت سرهمی که معکوس آن در پیمایش postorder باشد را پیدا می کنیم، اولین گره در این زوج ها، گره تک فرزندی می باشد.

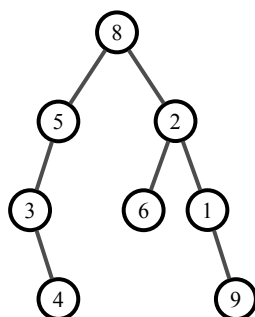
(VLR) preorder: 8 5 3 4 2 6 1 9

(LRV) postorder: 4 3 5 6 9 1 2 8

با توجه به دو عبارت بالا به زوج های زیر می رسم:

(5, 3), (3, 4), (1, 9)

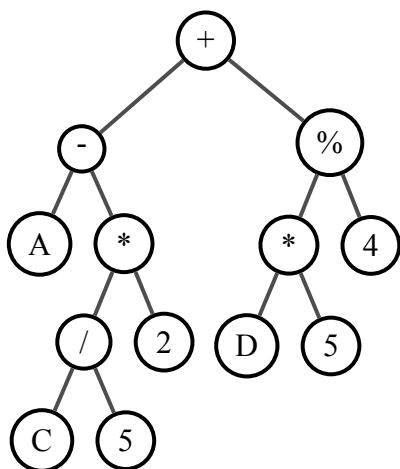
لذا در پیمایش preorder ۳ جفت پیدا کردیم که معکوس آن در postorder نیز می باشد.



۳.۴.۷ درخت عبارت

درخت عبارت از کاربرد های مهم درخت دودویی است که برای عبارت های ریاضی به کار می رود. برگ های درخت حروف و عدد هستند و گره های غیر برگ عملگر می باشند.

$$A - (C \div 3 \times 2) + (D \times 5 \% 4)$$



□ پیمایش inorder یک درخت عبارت، نگارش infix را تولید می کند.

□ پیمایش preorder یک درخت عبارت، نگارش prefix را تولید می کند.

□ پیمایش postorder یک درخت عبارت، نگارش postfix را تولید می کند.

۴.۴.۷ تعداد درخت های دودویی که می توان به n گره ساخت

فرض می کنیم هر یک از گره ها را به روش inorder از ۱ تا n شماره گذاری کرده ایم: اگر گره شماره i ریشه باشد، در آن صورت ۱- n گره در زیر درخت چپ ریشه و n-i گره در زیر درخت راست ریشه قرار دارد:

$$1, 2, 3, \dots, i-1, i, i+1, \dots, n$$

$$\underbrace{b_n}_{\text{تعداد درخت ها}} = \sum_{i=1}^n b_{i-1} b_{n-i}$$

$$b_0 = 1, b_1 = 1$$

۵.۴.۷ رسم درخت دودویی با داشتن پیمایش های پیشوندی و پسوندی

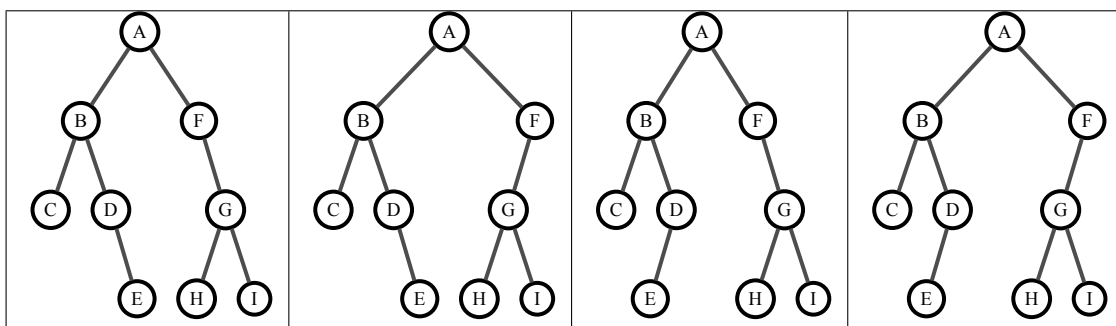
اگر پیمایش های پیشوندی و پسوندی یک درخت دودویی در دسترس باشند، محل گره های تک فرزندی را نمی توان مشخص کرد. بنابر این در صورت وجود گره های تک فرزندی، چندین درخت می توان ایجاد کرد که پیمایش preorder و postorder آنها باهم برابر باشند. این درخت ها برابر است با 2^k که در آن k تعداد گره های تک فرزندی است.

مثال ۱۰.۷ با توجه به preorder و postorder بیان شده، درخت را رسم کنید:

(VLR) preorder: A B C D E F G H I

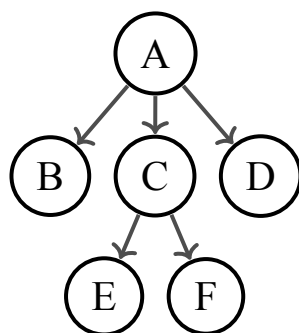
(LRV) postorder: C E D B H I G H A

پاسخ ۱۰.۷ با توجه به اینکه k تعداد گره های تک فرزندی است لذا $2^2 = 4$ حالت داریم که گره های تک فرزندی آن ها مشخص است.



۵.۷ درخت کلی (عمومی)

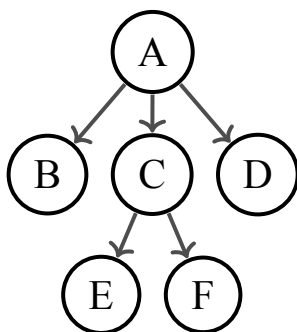
با فرض اینکه یک گره دارای یک فرزند باشد، آنگاه این فرزند در یک درخت دودویی با عنوان بچه چپ یا راست از هم متمایز می شوند، اما در یک درخت کلی تفاوتی بین آنها وجود ندارد. برای پیاده سازی درخت های کلی، آنها را به صورت یک درخت دودویی در می آوریم که به آن (درخت دودویی معادل) با درخت اصلی می گوئیم.



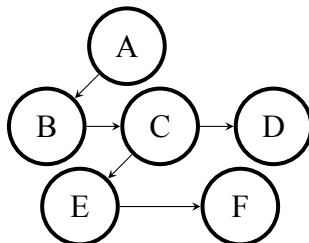
۱.۵.۷ مراحل تبدیل یک درخت به درخت دودویی معادل

برای تبدیل فوق باید مراحل زیر را به ترتیب انجام دهیم.

مرحله اول) در هر سطح باید کلیه گره های کنار هم (فرزندان یک پدر) را به یکدیگر متصل کنیم.



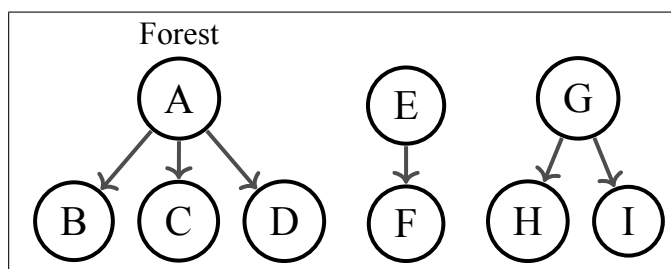
مرحله دوم) کلیه اتصالات گره را به گره پدر، به جز اتصال سمت چپ ترین فرزند را قطع می کنیم.



اولین فرزند هر گره در درخت اصلی، فرزند چپ آن در درخت دودویی معادل است و برادر سمت راست هر گره در درخت اصلی فرزند راست آن، در درخت معادل است.

۶.۷ جنگل Forest

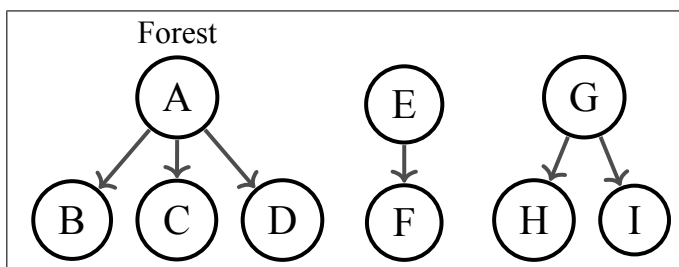
چند درخت کلی اگر کنار هم قرار گیرند، تشکیل یک جنگل می دهند.
به عبارت دیگر جنگل شامل n درخت مجزا است ($n \geq 0$)



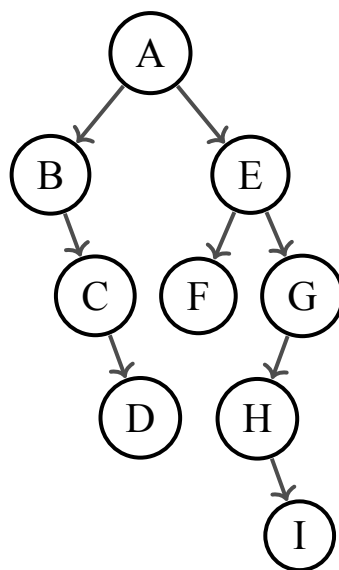
۱.۶.۷ تبدیل جنگل به درخت دودویی

برای تبدیل فوق باید مراحل زیر را به ترتیب انجام دهیم.

مرحله اول) هر درخت جنگل را به یک درخت دودویی تبدیل می کنیم.



مرحله دوم) درخت های دودویی را از طریق فرزند راست گره های ریشه به هم متصل می کنیم.



در درخت دودویی حاصل از جنگل، تعداد برگ ها، برابر تعداد درخت هایی است که جنگل را تشکیل داده اند.

۷.۷ پیاده سازی الگوریتم های مربوط به درخت

۱.۷.۷ ساختار یک گره یک درخت دودویی

```
1 struct node{int data;struct node* left; struct node* right;
    };
```

۲.۷.۷ تابع newnode

این تابع برای ایجاد گره های متعدد برای یک درخت می باشد.

```
1 struct node* newnode(int item){
2     struct node* node = (struct node*) malloc(sizeof(struct
    node)); (1)
3     node->data = item; (2)
4     node->left = Null; (3)
5     node->right = Null; (4)
6     return node;
7 }
```

(۱) با دستور malloc دریافت یک فضای خالی از حافظه به اندازه یک node را انجام می دهیم.

(۲) در قسمت data، node ایجاد شده، مقدار item را قرار بده.

(۳،۴) در دو قسمت left و right گره ایجاد شده، که برای آدرس دهی ماست Null را قرار بده.

۳.۷.۷ شمارش تعداد گره ها

این تابع آدرس ریشه یک درخت را می گیرد و تعداد گره های درخت را بر می گرداند.

```
1 unsigned int count(struct node* root){
2     if(root = Null)
3         return (0);
4     return (1 + count(root->left) + count(root->right));
5 }
```

این تابع به صورت بازگشتی شمارش تعداد گره های یک درخت دودویی را انجام می دهد.

۴.۷.۷ تشخیص برگ بودن

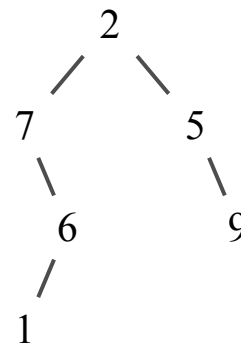
این تابع برای تشخیص برگ بودن می باشد، به تابع زیر یک گره به عنوان ورودی می‌دهیم و تشخیص می‌دهد که گره برگ است یا خیر.

```
1 bool isleaf(struct node* n){
2     if(n==Null)
3         return false;
4     if((n->left==Null) && (n->right==Null))
5         return true;
6     return false;
7 }
```

مثال ۱۱.۷ با توجه به الگوریتم‌های بیان شده درخت زیر را ایجاد کنید.

پاسخ ۱۱.۷ با توجه به درخت زیر داریم:

```
1 int main(void){
2     struct node* nr = Null;
3     struct node* root = newnode(2)
4     root->left = newnode(7);
5     root->right = newnode(5);
6     root->left->right = newnode(6);
7     root->left->right->left = newnode(1);
8     root->right->right = newnode(9);
9 }
```



۵.۷.۷ تشخیص کامل بودن درخت دودویی

تابع زیر تشخیص می‌دهد که درخت دودویی کامل است یا خیر.

```
1 bool f(struct node* root, unsigned int i, unsigned int n){
2     if(root == Null)
3         return (true);
4     if(i>n)
5         return false;
6     return(f(root->left, 2*i, n) && f(root->right, 2i+1, n)
7 )
7 }
```

این تابع نیز به صورت بازگشتی بررسی می‌کند که یک درخت کامل است یا خیر.

۶.۷.۷ حذف گره های تک فرزندی

تابع زیر گره های تک فرزندی یک درخت را به صورت بازگشتی حذف می کند.

```
1 struct node* remove(struct node* root){
2     if(root==NULL) return Null;
3     root->left = remove(root->left);
4     root->right = remove(root->right);
5     if((root->left==Null) && (root->right==Null))
6         return root;
7     if(root->left==Null)
8         {struct node* n = root->right;free(root):return n;}
9     if(root->right==Null)
10        {struct node* n = root->left;free(root):return n;}
11 }
```

۷.۷.۷ پیاده سازی درخت در صف

ایجاد صف برای پیاده سازی درخت

```
1 struct node** createQueue(int *front, int *rear){
2     struct node** queue = (struct node**)malloc(sizeof(
3     struct node*)*Max);
4     *front = *rear = 0;
5     return queue;
6 }
```

دریافت گره (مقدار) برای درج در صف

```
1 void enqueue(struct node **queue, int *rear,struct node *
2     newnode){
3     queue[*rear] = newnode;
4     (*rear)++;
5 }
```

حذف گره از صف (حذف گره)

```
1 struct node* dequeue(struct node **queue, int *front){
2     (*front)++;
3     return queue[*front-1]
4 }
```

۸ گراف

گراف $G = (V, E)$ مجموعه ای از راس و یال است.
 V : گره، مجموعه محدود و غیر تهی از رئوس E : یال، مجموعه از زوج رئوس

انواع گراف

۱. جهت دار:

ترتیب گره ها در زوج مرتب اهمیت دارد.

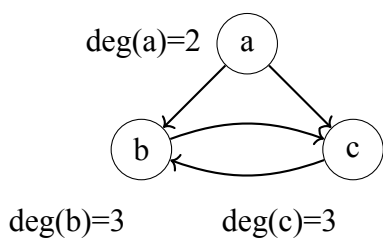
۲. غیر جهت دار:

گرافی که در آن جای دو راس در مجموعه یال ها اهمیت نداشته باشد.

درجه یک گره به معنی یال هایی است که به یک گره برخورد می کند می باشد.
 برای محاسبه تعداد کل یال ها همه درجه ها را با هم جمع کرده و تقسیم برای ۲ میکنیم .
 به طور مثال در زیر دو گراف داریم که به ترتیب بدون جهت و جهت دارد می باشند.
 استفاده از مجموعه به معنی این است که گراف بدون جهت می باشد و استفاده از پرانتز برای یال های یا به معنی جهت دار بودن گراف می باشد.

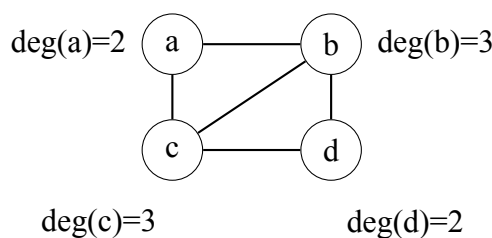
$$V = \{a, b, c\}$$

$$E = \{(a, c), (a, b), (b, c), (c, b)\}$$



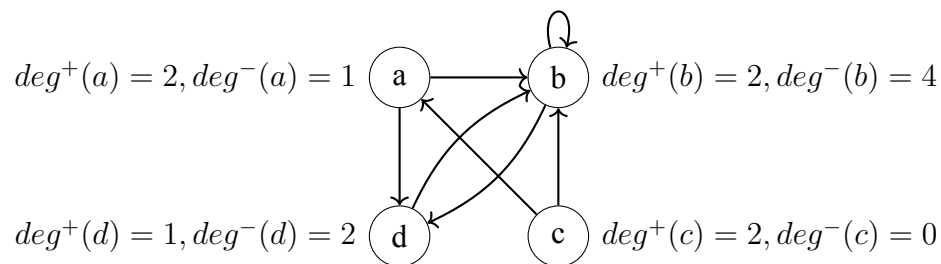
$$V = \{a, b, c, d\}$$

$$E = \{\{a, b\}, \{a, c\}, \{b, c\}, \{b, d\}, \{c, d\}\}$$



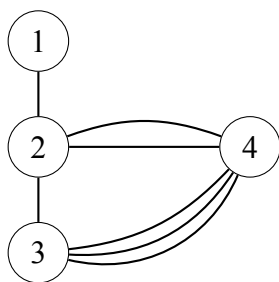
می توانیم برای گراف جهت دار درجه ها را نیز تفکیک کنیم یعنی داشته باشیم :
 deg^+ اشاره به یال خروجی از یک گره
 deg^- اشاره به یال های ورودی یک گره

در حالت deg اشاره به درجه کل گره می باشد، یعنی جمع تمام گره های ورودی و خروجی از یک گره.



گراف چند گانه

گرافی که در آن یال های چند گانه مجاز باشد را گراف چند گانه یا (multiEdge) می گویند.



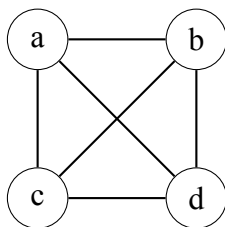
گراف کامل

گراف بدون جهتی که همه یال های آن رسم شده باشند.

درجه هر یک از گره ها: $n - 1$

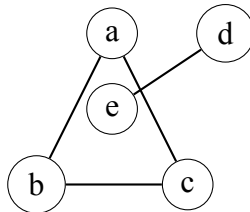
تعداد یال ها: $\frac{n(n-1)}{2}$

به طور مثال گراف زیر ۴ راس دارد که هر راس به راس دیگر وصل است:



گراف هم بند

گرافی که یک مسیر بین هر دو گره آن وجود داشته باشد.
به طور مثال شکل زیر یک گراف ناهمبند است:



گراف هم بند قوی:

گرافی جهت داری که برای هر زوج گره u, v ، هم یک مسیر از u به v و هم یک مسیر از v به u وجود داشته باشد.

۱.۸ نمایش گراف

۱. ماتریس همجواری (همسایگی) Adjacency Matrices

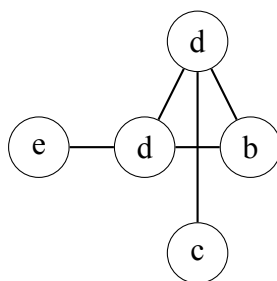
۲. لیست همجواری Adjacency List

ماتریس همجواری

با ذکر دو مثال ماتریس همجواری را برای دو گراف جهت دار و بدون جهت شرح می‌دهیم.

مثال ۱.۸ نمایش ماتریس همجواری گراف بدون جهت زیر:

پاسخ ۱.۸ با توجه به گراف روبه رو داریم:



	a	b	c	d	e
a	0	1	1	1	0
b	1	0	0	1	0
c	1	0	0	0	0
d	1	1	0	0	1
e	0	0	0	1	0

مرحله اول) ماتریس ایجاد می‌کنیم به اندازه سطر و ستون تعداد گره‌های گراف و سپس به بررسی وضعیت یال‌های هر گره می‌پردازیم.

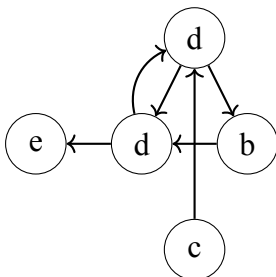
مرحله دوم) گره a با سه گره b, c, d همجوار است لذا سطر مربوط در ماتریس را ۱ قرار داده و مابقی خانه ۰ می‌شوند.

گره b با گره a, d همجوار است لذا سطرهای مربوطه در ماتریس ۱ می‌شوند و مابقی درایه‌ها ۰.

به همین ترتیب عمل فوق را برای هر گره تکرار کرده تا جدول کامل شود و به ماتریس همجواری برسیم.

مثال ۲.۸ نمایش ماتریس همجواری گراف جهت دار زیر:

پاسخ ۲.۸ با توجه به گراف روبه رو داریم:



	a	b	c	d	e
a	0	1	0	1	0
b	0	0	0	1	0
c	1	0	0	0	0
d	1	0	0	0	1
e	0	0	0	0	0

مرحله اول) ماتریس ایجاد می کنیم به اندازه سطر و ستون تعداد گره های گراف و سپس به بررسی وضعیت یال های هر گره می پردازیم ، با این تفاوت که مبانی ۱ شدن درایه ماتریس در این است که از گره مد نظر ما باید یالی خارج شده باشد. و در غیر اینصورت یعنی اگر یالی وجود نداشته باشد یا یال به گره وارد شده باشد در درایه مربوطه ۰ قرار می دهیم.

مرحله دوم) گره a با سه گره b, d همجوار است لذا سطر مربوط در ماتریس را ۱ قرار می دهیم و مابقی درایه ها ۰ می شوند. گره b با گره d همجوار است لذا درایه مربوطه در ماتریس ۱ می شوند و مابقی درایه ها ۰ می شوند.

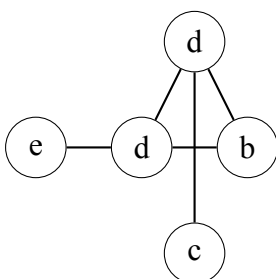
به همین ترتیب عمل فوق را برای هر گره تکرار کرده تا جدول کامل شود و به ماتریس همجواری برسیم.

لیست همجواری

لیست همجواری را نیز همانند ماتریس همجواری با ذکر دو مثال شرح می دهیم .

مثال ۳.۸ نمایش همجواری گراف بدون جهت زیر:

پاسخ ۳.۸ با توجه به گراف روبه رو داریم:



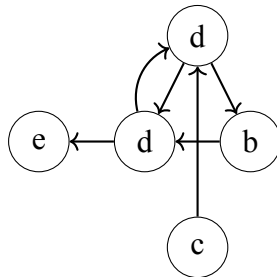
a		→	b	→	d	→	c	/
b		→	a	→	d	/		
c		→	a	/				
d		→	e	→	b	→	a	/
e		→	d	/				

مرحله اول) برای هر گره موجود در گراف لیست ای در نظرم می گیرم.
سپس لیست همجواری بدین صورت است که گره مد نظر به هر گره دیگری یال داشت، آنرا در یک خانه لیست مربوط به گره می نویسیم.

مرحله دوم) گره a با سه گره c, b, d همجوار است، لذا درون آرایه a نام این سه گره را درج میکنیم.
گره b با گره های a, d همجوار است لذا در لیست مربوط به b این اسامی را قرار می دهیم.
کار را به همین ترتیب ادامه داده یا همه لیست همه گره ها کامل شود.

مثال ۴.۸ نمایش لیست همجواری گراف جهت دار :

پاسخ ۴.۸ با توجه به گراف روبه رو داریم:



a		→	b	/		
b		→	d	/		
c		→	a	/		
d		→	e	→	a	/
e	/					

نحوه ایجاد لیست دقیقاً همانند مثال بالاست، فقط در زمانی که یک یال از گره مورد نظر خارج می شود، باید در لیست آن گره نوشته شود.

۲.۸ پیمایش در گراف ها

پیمایش گراف ها به صورت زیر است:

۱. عمقی (پیمایش اول - عمق) DFS : Depth First Search

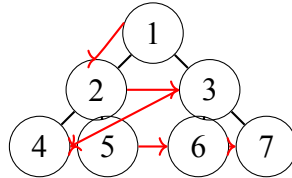
۲. سطحی (پیمایش اول - عرض) BFS : Breadth First Search

در پیمایش DFS از پشته و در پیمایش BFS از صف کمک می گیریم.

پیمایش سطحی

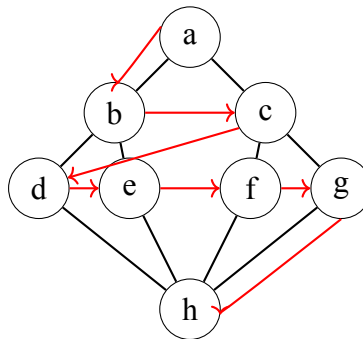
با شروع از یک گره ابتدا گره را وارد صف کرده، سپس کلیه گره های مجاور با آن (فرزندان گره از چپ به راست) را در صف درج می کنیم.

حال عنصر بعدی را از صف حذف کرده و گره های مجاور با آن را درج می کنیم.
این عمل را ادامه می دهیم تا همه گره ها پیمایش شوند.
پیشمایش سطحی منحصر به فرد نیست.
نحوه حرکت در پیمایش سطحی :



مثال ۵.۸ یکی از پیمایش های BFS گراف زیر را مشخص کنید.

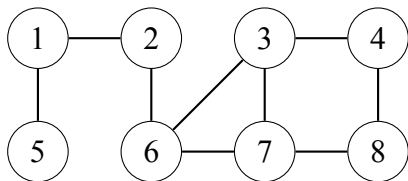
پاسخ ۵.۸ BFS : a b c d e f g h



مثال ۶.۸ یکی از پیمایش های BFS گراف زیر با شروع از گره شماره ۲ مشخص کنید.

پاسخ ۶.۸ با ایجاد یک صف پیمایش BFS را می توان محاسبه کرده، لذا حاصل پیشمایش BFS به صورت زیر خواهد بود

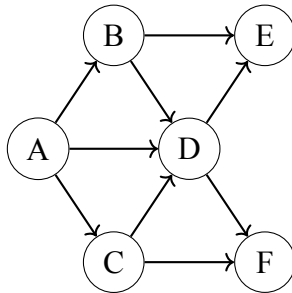
BFS : 2 6 1 3 7 5 4 8



2		
6	1	
1	3	7
3	7	5
7	5	4
5	4	8
4	8	
8		

مثال ۷.۸ سه پیمایش BFS گراف جهت دار زیر را بنویسید.

پاسخ ۷.۸ برای نوشتن پیمایش های متفاوت BFS کافیست هر دفعه مسیر های سطری متفاوتی را برویم تا به BSF های متفاوتی برسیم.



BFS = A B C D E F

BFS = A D B C F E

BFS = A D B C E F

پیمایش BFS می تواند منحصر به فرد نباشد و شروع از هر گره دلخواه باشد. فقط در زمان نوشتن پیمایش BFS گراف جهت دار دقت شود که حرکت بین گره ها از طریق جهت یال ها انجام می شود.

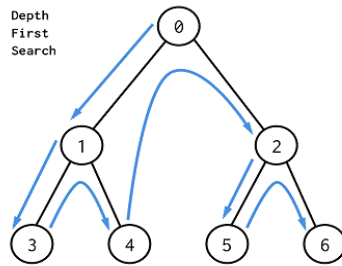
تابع پیمایش سطحی (BFS)

```

1 bfs(v){
2     front = rear = Null;
3     cout<<v;
4     visited[v] = True;
5     addq(&front, &rear, v);
6     while(front){
7         v = deleteq(&front);
8         for(w=graph[v];w;w=w->link);
9         if(!visited[w->vertex]){
10            cout<<w->vertex;
11            addq(&front, &rear, w->vertex);
12            visited[w->vertex] = True;}
13    }}
  
```

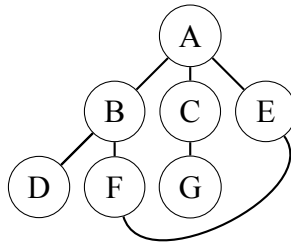
پیمایش عمقی

با شروع از یک گره، آنرا ملاقات کرده و سپس کلیه گره های سمت چپ را تا آخرین عمق پیمایش میکنیم .
اگر در پایین رفتن های متوالی، گره ای ملاقات شود که هیچ گره همجواری نداشته باشد یا با تمام گره های همجوار آن ملاقات شده باشد، برگشت به یک سطح بالاتر انجام می گیرد و روند فوق تکرار می شود.
پیمایش عمقی منحصر به فرد نیست.



مثال ۸.۸ یکی از پیمایش های DFS گراف زیر را مشخص کنید.

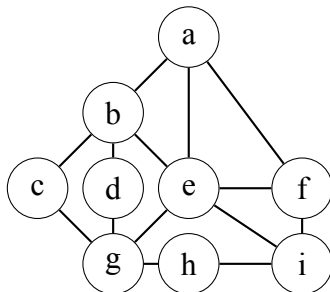
پاسخ ۸.۸ با توجه به گراف داریم:



DFS = A B D F E C G

مثال ۹.۸ یکی از پیمایش های DFS گراف زیر را مشخص کنید.

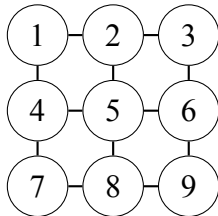
پاسخ ۹.۸



DFS = a b c g d e f i h

مثال ۱۰.۸ سه پیمایش DFS برای گراف زیر را مشخص کنید.

پاسخ ۱۰.۸



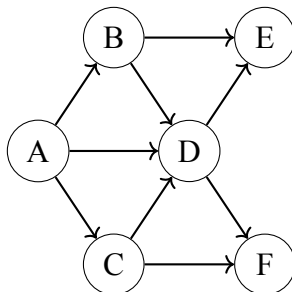
DFS = 1 2 3 6 9 8 7 4 5

DFS = 1 2 3 6 5 4 7 8 9

DFS = 1 4 7 8 9 6 5 2 3

مثال ۱۱.۸ سه پیمایش BFS گراف جهت دار زیر را بنویسید.

پاسخ ۱۱.۸



DFS = A B E D F C

DFS = A B D F E C

DFS = A D E F B C

تابع پیمایش عمقی DFS

این تابع به دلیل بازگشتی بودن از پشته استفاده می کند.

```

1 dfs(v){
2     visited[v] = True;
3     cout<<v;
4     for(w=graph[v];w;w=w->link)
5         if(!visited[w->vertex])
6             dfs(w->vertex);
7 }
```

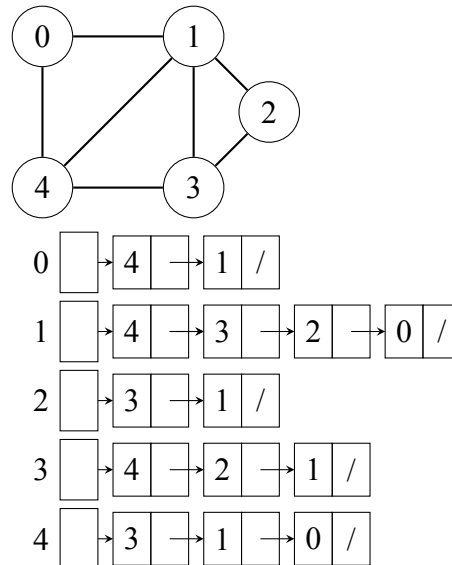
۳.۸ پیاده سازی گراف

پیاده سازی گراف با لیست همجواری

```

1  int main(){
2      int v = 5;
3      struct Graph* g;
4      g = createGraph(v);
5      addEdge(g, 0, 1);
6      addEdge(g, 0, 4);
7      addEdge(g, 1, 2);
8      addEdge(g, 1, 3);
9      addEdge(g, 1, 4);
10     addEdge(g, 2, 3);
11     addEdge(g, 3, 4);
12     printGraph(g)
13 }

```



تعریف استراکچر ها

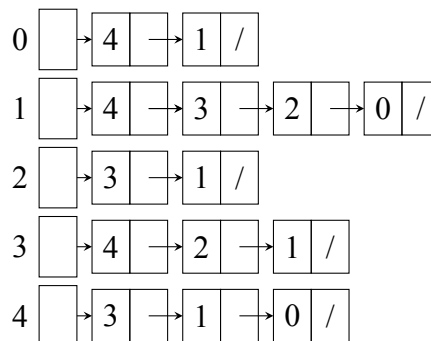
```

1 struct Node{
2     int data;
3     struct Node* next;
4 }

1 struct AdjList{
2     struct Node *head;
3 }

1 struct Graph{
2     int v;
3     struct AdjList* array;
4 }

```

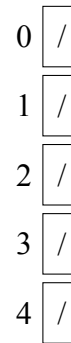


ایجاد یک گره

```
1 struct Node* newNode(int a){
2     struct Node* n;
3     n = malloc(sizeof(struct Node));
4     n->data = Null;
5     n->next = Null;
6     return n;
7 }
```

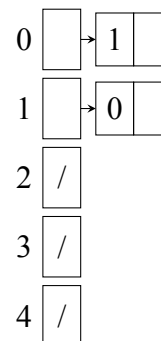
ساخت گراف

```
1 struct Graph* createGraph(int v){
2     struct Graph* g;
3     g= malloc(sizeof(structGraph));
4     g->v = v;
5     g->array = malloc(v * sizeof(
        structAdjList));
6     int i;
7     for(i=0;i<v;++i){
8         g->array[i].head = Null;}
9     retrun g;
10 }
```



اضافه کردن یال

```
1 void addEdge(struct Graph* g, int s
  , int d){
2
3     // Add an edge from source to
    dest.
4
5     struct Node* n = newNode(d);
6     n->next = g->array[s].head;
7     g->array[s].head = n
8
9     // Since the graph is
    undirected, add an edge from dest
    to source also
10
11     n = newNode(s);
12     n->next = g->array[d].head;
13     g->array[d].head = n
14 }
```

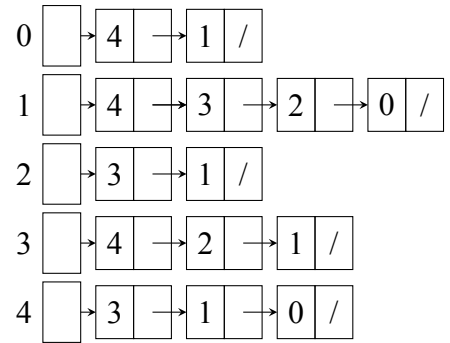


چاپ گراف

```

1 void printGraph(struct Graph* g)
2 {
3     int i;
4     struct Node* p;
5     for(i=0;i<g->v;++i)
6     {
7         p = g->array[i].head;
8         printf("vertex %d:",i);
9         while(p){
10             printf("-> %d", p->data
11         );
12             p = p->next;
13         }
14         printf("\n");
15     }

```



On screen:

```

vertex0: -> 4 -> 1
vertex1: -> 4 -> 3 -> 2 -> 0
vertex2: -> 3 -> 1
vertex3: -> 4 -> 2 -> 1
vertex4: -> 3 -> 1 -> 0

```


۹ مرتب سازی

الگوریتم های مرتب سازی را از نظر نحوه مرتب سازی می توان به دو دسته تقسیم کرد.

۱. مرتب سازی مبتنی بر مقایسه (Comparison based):

با مقایسه بین کلیه عناصر، ترتیب نسبی آنها پیدا می شود.

۲. مرتب سازی غیر مقایسه ای یا خطی (Non-Comparison based):

بدون مقایسه کلیه عناصر با هم، عمل مرتب سازی انجام می شود.

الگوریتم های مرتب سازی مقایسه ای:

□ حبابی (Bubble)

□ درجی (Insertion)

□ سریع (Quick)

□ درختی (Tree)

□ انتخابی (Selection)

□ ادغامی (Merge)

□ هیپ (Heap)

□ شل (Shell)

الگوریتم های مرتب سازی غیر مقایسه ای:

□ شمارشی (Count)

□ سطلی (Bucket sort)

□ مبنایی (Radix sort)

□ لانه کبوتری

در این درس فقط به بررسی سه الگوریتم حبابی، درجی و انتخابی خواهیم پرداخت.

۱.۹ مرتب سازی حبابی

در این روش با چند بار حرکت در طول آرایه، یک عنصر با عنصر بعدی مقایسه شده و در صورت لزوم جابه جا می شود. به عنوان مثال برای بررسی عملکرد مرتب سازی حبابی آرایه ای به صورت (8, 2, 4, 1, 5) داریم، مراحل مرتب سازی حبابی برای این آرایه به صورت زیر است.

مقدار آرایه به عنوان ورود به الگوریتم داده می شود.	(8, 2, 4, 1, 5)
در گام اول، عدد ۵ و ۱ با هم مقایسه شده و در نتیجه جای این مقدار با هم عوض می شود.	(8, 2, 4, 5, 1)
در گام دوم، دو مقدار ۵ و ۴ با هم مقایسه می شوند و در نتیجه جای این دو مقدار در آرایه نیز با هم عوض می شود.	(8, 2, 5, 4, 1)
در گام سوم، عدد ۲ تا ۵ مقایسه شده و به دلیل بزرگتر بودن ۵ جای این دو مقدار نیز با هم عوض می شود.	(8, 2, 5, 4, 1)
در نهایت دو مقدار ۸ و ۵ را با هم مقایسه کرده که در نتیجه نیاز به تعویض نداریم.	(8, 2, 5, 4, 1)
حال باید به مرحله دوم مرتب سازی برویم (فاز ۲)	
در این مرحله نیز دقیقاً همانند مرحله اول، مقایسه عناصر را با هم مقایسه میکنیم.	(8, 2, 5, 4, 1)
با تعویض خانه دو عنصر ۲ و ۴ با هم به فرم نهایی رسیده و مرتب سازی به اتمام خواهد رسید.	(8, 2, 5, 4, 1)

در هر مرحله ای که در مرتب سازی حبابی طی می شود، یک عنصر از آخر لیست برای مقایسه نشدن، حذف می شود. (لیست تغییر نمی کند بلکه بازه مقایسه ماست که کوچک می شود).

همچنین با ایجاد یک **flag** می توانیم این الگوریتم را هوشمند کنیم، در صورتیکه عمل **swap** در مرحله ای انجام نشد، دیگر ادامه نده.

مثال ۱.۹ با کمک مرتب سازی حبابی آرایه زیر را مرتب کنید: (8, 4, 3, 2)

مرحله مرتب سازی وضعیت عناصر در آرایه

1) (8, 4, 3, 2)

(4, 8, 3, 2)

(4, 3, 8, 2)

(4, 3, 2, 8)

پاسخ ۱.۹

2) (3, 4, 2, 8)

(3, 2, 4, 8)

3) (2, 3, 4, 8)

الگوریتم مرتب سازی حبابی

```
1 void bubblesort(int arr[], int n){
2     int i,j;bool swapped;
3     for(i=0;i<n-1;i++){
4         swapped = false;
5         for(j=0;j<n-i-1;j++){
6             if(arr[j]>arr[j+1]){
7                 swap(&arr[j], &arr[j+1]);
8                 swapped = True;
9             }
10        if(swapped == false) break;
11    }
12 }
```

۲.۹ مرتب سازی انتخابی

قرار دادن کوچکترین عنصر آرایه در مکان اول آرایه
قرار دادن کوچکترین عنصر دوم آرایه در مکان دوم آرایه و به همین ترتیب برای تمام عناصر.
به طور مثال برای آرایه (10, 30, 17, 12, 1, 21, 15) مرتب سازی انتخابی به صورت زیر می باشد:

10	30	17	12	1	21	15
<u>1</u>	30	17	12	10	21	15
<u>1</u>	<u>10</u>	17	12	30	21	15
<u>1</u>	<u>10</u>	<u>12</u>	17	30	21	15
<u>1</u>	<u>10</u>	<u>12</u>	<u>15</u>	30	21	17
<u>1</u>	<u>10</u>	<u>12</u>	<u>15</u>	<u>17</u>	21	30
<u>1</u>	<u>10</u>	<u>12</u>	<u>15</u>	<u>17</u>	<u>21</u>	30

اگر در روش انتخابی به جای کوچکترین عنصر، بزرگترین عنصر را پیدا کنیم، آنگاه لیست ما به صورت نزولی در خواهد آمد.
برای n عنصر $n-1$ مرحله در مرتب سازی انتخابی داریم.
الگوریتم حبابی و انتخابی هر دو دارای مرتبه اجرایی n^2 می باشد.

الگوریتم مرتب سازی انتخابی

```

1 void selectionSort(int arr[], int n){
2     int i,j,m;
3     for(i=0;i<n-1;i++){
4         m = i;
5         for(j=j+1;j<n;j++){
6             if(arr[j]<arr[m])
7                 m = j;
8         }
9         swap(&arr[m], &arr[i]);
10    }
11 }

```

۳.۹ مرتب سازی درجی

با فرض اینکه عدد اول در لیست در جای درستی قرار گرفته است عدد دوم را با آن مقایسه می کنیم، اگر کوچک تر بود جابه جا شود ، سپس عدد سوم را با دو عددی که با هم مقایسه کرده ایم بررسی میکنیم و به همین ترتیب.

<u>8</u>	5	3	21	12	11	6	2
<u>5</u>	<u>8</u>	3	21	12	11	6	2
<u>3</u>	<u>5</u>	<u>8</u>	21	12	11	6	2
<u>3</u>	<u>5</u>	<u>8</u>	<u>21</u>	12	11	6	2
<u>3</u>	<u>5</u>	<u>8</u>	<u>12</u>	<u>21</u>	11	6	2
<u>3</u>	<u>5</u>	<u>8</u>	<u>11</u>	<u>12</u>	<u>21</u>	6	2
<u>3</u>	<u>5</u>	<u>6</u>	<u>8</u>	<u>11</u>	<u>12</u>	<u>21</u>	2
<u>2</u>	<u>3</u>	<u>5</u>	<u>6</u>	<u>8</u>	<u>11</u>	<u>12</u>	<u>21</u>

```
1 void insertionSort(int arr[], int n){
2     int i,key,j;
3     for(i=1;i<n;i++){
4         key = arr[i]
5         j = i-1;
6         while(j>=0 && arr[j]>key){
7             arr[j+1] = arr[j];
8             j = j-1;
9         }
10        arr[j+1] = key;
11    }
12 }
```